ECE 303 – ECE Laboratory

Sofia D’Alessandro

Table of Contents

Subject: Assignment 1 – UART Communications

To: Christopher Peters, PhD.
Teaching Professor, Electrical and Computer Engineering Department
Drexel University

From: Sofia D'Alessandro
Undergraduate, College of Engineering
Drexel University

Purpose

The purpose of this memo is to demonstrate serial communication between Python and the Arduino using the UART, or Universal Asynchronous Receiver-Transmitter. Communication is done with the computer using the USB-C connection, or Serial port. This communication is done by encoding data as JSON strings and then sending, receiving, and unpacking them in a way that each application can understand.

Methodology/Approach

In conducting the lab, the first step was to physically build the circuit using a solderless breadboard, one red LED, one yellow LED, one green LED, blue LED, jumper wires, four 330-ohm resistors, and the Arduino Uno R4 WiFi. The same exact circuit was created four times, each with a different color LED. Each LED was connected to a specific digital pin at its anode. Its cathode was connected to a specific analog pin.

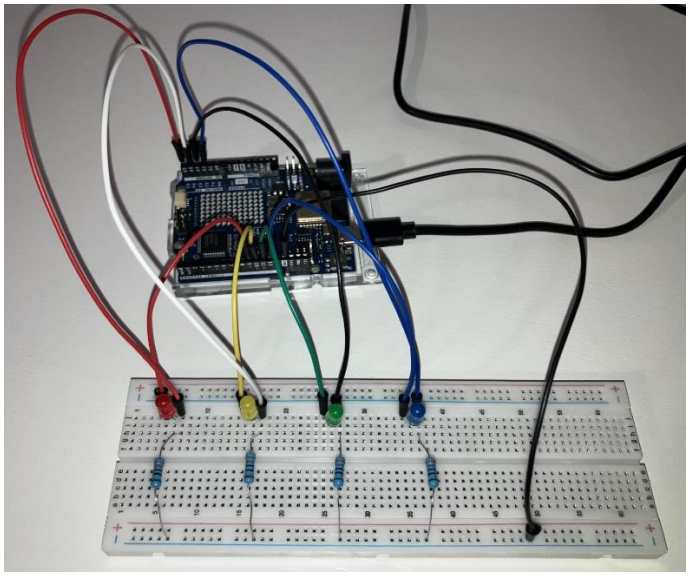
After the circuit was built, the next step was to begin working in the two applications that would be communicating with each other, Python and Arduino IDE. The programming was conducted simultaneously to follow the flow of the data. With the LEDs connected to their respective digital and analog pins, the pins could be initialized in the IDE. Communication with the serial port was also initialized. A connected to the serial port was also established on the Python side. In Python, each combination was initialized and then convert into a JSON-formatted string. In the main part of the program, the function to send the JSON string was called for each combination. In the IDE, if data was sent, it was read. The condition for each LED was checked and the LED was turned on or off based on that condition. The analogRead() function was then used to read and store the output voltages of the LEDs. These output voltages were converted to JSON-formatted strings and send back to Python. A function to read the data was also called in the main part of the Python program. The JSON strings were converted to a dictionary so that Python could interpret the data. The exit voltages were then calculated and outputted.

Results

The project was successful. The Arduino was able to read and interpret the JSON strings sent from Python to operate on the LEDs so that the combinations of the LEDs being on or off were correctly demonstrated when required. The Arduino was also able to read the exit voltages from each LED during each combination and send them to Python as JSON strings. The analogRead()

function, which was used to determine the exit voltages, captured a value from the analog pins. This value was not actually the voltage, but it represented it. These values were sent to Python in a dictionary where the exit voltages could then be calculated. The exit voltages were quite reasonable. When they were set to “off,” the exit voltages of all the LEDs were obviously 0 V but when they were set to “on,” the exit voltages of the red, yellow, and green LEDs were about 2.5 V while the exit voltage of the blue LED was about 1.8 V. It makes sense that the exit voltage of the blue LED would be less than that of the other LEDs even though they were all being powered by 5 V because blue LEDs typically require a higher forward voltage to emit light due to it having a shorter wavelength.

Picture of hardware:



Output of Python code:

Connected to COM3 at 9600 baud
Combination 1: red off, yellow on, green on, blue on
Sent: {"red": "off", "yellow": "on", "green": "on", "blue": "on"}
Received: {"red":0,"yellow":516,"green":504,"blue":361}
Exit voltage of red LED: 0.0V
Exit voltage of yellow LED: 2.5219941348973607V
Exit voltage of green LED: 2.463343108504399V
Exit voltage of blue LED: 1.7644183773216033V

Combination 2: red on, yellow off, green on, blue off
Sent: {"red": "on", "yellow": "off", "green": "on", "blue": "off"}
Received: {"red":503,"yellow":0,"green":505,"blue":0}
Exit voltage of red LED: 2.458455522971652V
Exit voltage of yellow LED: 0.0V
Exit voltage of green LED: 2.4682306940371457V
Exit voltage of blue LED: 0.0V

Combination 3: red on, yellow off, green off, blue on

Sent: {"red": "on", "yellow": "off", "green": "off", "blue": "on"}
Received: {"red":510,"yellow":0,"green":0,"blue":363}
Exit voltage of red LED: 2.4926686217008798V
Exit voltage of yellow LED: 0.0V
Exit voltage of green LED: 0.0V
Exit voltage of blue LED: 1.774193548387097V

Combination 4: red on, yellow on, green on, blue on
Sent: {"red": "on", "yellow": "on", "green": "on", "blue": "on"}
Received: {"red":510,"yellow":515,"green":501,"blue":361}
Exit voltage of red LED: 2.4926686217008798V
Exit voltage of yellow LED: 2.517106549364614V
Exit voltage of green LED: 2.4486803519061584V
Exit voltage of blue LED: 1.7644183773216033V

Combination 5: red off, yellow off, green off, blue off
Sent: {"red": "off", "yellow": "off", "green": "off", "blue": "off"}
Received: {"red":0,"yellow":0,"green":0,"blue":0}
Exit voltage of red LED: 0.0V
Exit voltage of yellow LED: 0.0V
Exit voltage of green LED: 0.0V
Exit voltage of blue LED: 0.0V

Appendix

Python code:

```
import serial, json, time

# Configure the serial port
port = 'COM3'
baudrate = 9600
try:
    ser = serial.Serial(port, baudrate, timeout=1)
    print(f"Connected to {port} at {baudrate} baud")
except serial.SerialException as e:
    print(f"Error opening serial port: {e}")
    exit()

# Function to send data
def send_data(data):
    ser.write(data.encode())
    print(f"Sent: {data}")

# Initialize combinations
combination1 = {"red": "off", "yellow": "on", "green": "on", "blue": "on"}
combination2 = {"red": "on", "yellow": "off", "green": "on", "blue": "off"}
```

```
combination3 = {"red": "on", "yellow": "off", "green": "off", "blue": "on"}
combination4 = {"red": "on", "yellow": "on", "green": "on", "blue": "on"}
combination5 = {"red": "off", "yellow": "off", "green": "off", "blue": "off"}

# Convert combinations to JSON
json_combination1 = json.dumps(combination1)
json_combination2 = json.dumps(combination2)
json_combination3 = json.dumps(combination3)
json_combination4 = json.dumps(combination4)
json_combination5 = json.dumps(combination5)

# Function to read data
def read_data():
    if ser.in_waiting > 0:
        data = ser.readline().decode('utf-8').rstrip()
        print(f"Received: {data}")
        return data
    return None

# Main
try:
    # Combination 1
    print("Combination 1: red off, yellow on, green on, blue on")
    send_data(json_combination1) # Send data
    time.sleep(1) # Wait for 1 second
    response = read_data() # Read data
    data_object = json.loads(response) # Convert JSON string to Python object
    # Calculate the exit voltage of each LED
    red_voltage = (data_object["red"] / 1023.0) * 5
    yellow_voltage = (data_object["yellow"] / 1023.0) * 5
    green_voltage = (data_object["green"] / 1023.0) * 5
    blue_voltage = (data_object["blue"] / 1023.0) * 5
    # Print the exit voltage of each LED
    print(f"Exit voltage of red LED: {red_voltage}V")
    print(f"Exit voltage of yellow LED: {yellow_voltage}V")
    print(f"Exit voltage of green LED: {green_voltage}V")
    print(f"Exit voltage of blue LED: {blue_voltage}V")
    print("\n") # Print a new line
    time.sleep(10) # Wait for 10 seconds

    # Combination 2
    print("Combination 2: red on, yellow off, green on, blue off")
    send_data(json_combination2)
    time.sleep(1)
    response = read_data()
    data_object = json.loads(response)
```

```
red_voltage = (data_object["red"] / 1023.0) * 5
yellow_voltage = (data_object["yellow"] / 1023.0) * 5
green_voltage = (data_object["green"] / 1023.0) * 5
blue_voltage = (data_object["blue"] / 1023.0) * 5
print(f"Exit voltage of red LED: {red_voltage}V")
print(f"Exit voltage of yellow LED: {yellow_voltage}V")
print(f"Exit voltage of green LED: {green_voltage}V")
print(f"Exit voltage of blue LED: {blue_voltage}V")
print("\n")
time.sleep(10)

# Combination 3
print("Combination 3: red on, yellow off, green off, blue on")
send_data(json_combination3)
time.sleep(1)
response = read_data()
data_object = json.loads(response)
red_voltage = (data_object["red"] / 1023.0) * 5
yellow_voltage = (data_object["yellow"] / 1023.0) * 5
green_voltage = (data_object["green"] / 1023.0) * 5
blue_voltage = (data_object["blue"] / 1023.0) * 5
print(f"Exit voltage of red LED: {red_voltage}V")
print(f"Exit voltage of yellow LED: {yellow_voltage}V")
print(f"Exit voltage of green LED: {green_voltage}V")
print(f"Exit voltage of blue LED: {blue_voltage}V")
print("\n")
time.sleep(10)

# Combination 4
print("Combination 4: red on, yellow on, green on, blue on")
send_data(json_combination4)
time.sleep(1)
response = read_data()
data_object = json.loads(response)
red_voltage = (data_object["red"] / 1023.0) * 5
yellow_voltage = (data_object["yellow"] / 1023.0) * 5
green_voltage = (data_object["green"] / 1023.0) * 5
blue_voltage = (data_object["blue"] / 1023.0) * 5
print(f"Exit voltage of red LED: {red_voltage}V")
print(f"Exit voltage of yellow LED: {yellow_voltage}V")
print(f"Exit voltage of green LED: {green_voltage}V")
print(f"Exit voltage of blue LED: {blue_voltage}V")
print("\n")
time.sleep(10)

# Combination 5
```

```
    print("Combination 5: red off, yellow off, green off, blue off")
    send_data(json_combination5)
    time.sleep(1)
    response = read_data()
    data_object = json.loads(response)
    red_voltage = (data_object["red"] / 1023.0) * 5
    yellow_voltage = (data_object["yellow"] / 1023.0) * 5
    green_voltage = (data_object["green"] / 1023.0) * 5
    blue_voltage = (data_object["blue"] / 1023.0) * 5
    print(f"Exit voltage of red LED: {red_voltage}V")
    print(f"Exit voltage of yellow LED: {yellow_voltage}V")
    print(f"Exit voltage of green LED: {green_voltage}V")
    print(f"Exit voltage of blue LED: {blue_voltage}V")
    print("\n")
    time.sleep(10)

except KeyboardInterrupt:
    # Close the serial port
    ser.close()
    print("Serial port closed.")
```

Arduino code:

```
#include <ArduinoJson.h>

// initialize pins for each LED
int red = 9;
int yellow = 10;
int green = 11;
int blue = 12;

void setup() {
  // setup
  Serial.begin(9600);
  Serial.setTimeout(500);

  pinMode(red, OUTPUT);
  pinMode(yellow, OUTPUT);
  pinMode(green, OUTPUT);
  pinMode(blue, OUTPUT);
}

void loop() {
  // read JSON data if available
  if (Serial.available() > 0) {
```

```
  StaticJsonDocument<200> combination;
  StaticJsonDocument<200> output;
  String incoming = Serial.readString();
  deserializeJson(combination, incoming);

  // check condition for each LED, turn on/off LED as necessary and then read exit voltage
  if (combination["red"] == "on") {
    digitalWrite(red, HIGH);  // turn the LED on (HIGH is the voltage level)
    output["red"] = analogRead(A5); // read exit voltage and store in output
  } else {
    digitalWrite(red, LOW); // turn the LED off (LOW is the voltage level)
    output["red"] = analogRead(A5);
  }

  if (combination["yellow"] == "on") {
    digitalWrite(yellow, HIGH);
    output["yellow"] = analogRead(A4);
  } else {
    digitalWrite(yellow, LOW);
    output["yellow"] = analogRead(A4);
  }

  if (combination["green"] == "on") {
    digitalWrite(green, HIGH);
    output["green"] = analogRead(A3);
  } else {
    digitalWrite(green, LOW);
    output["green"] = analogRead(A3);
  }

  if (combination["blue"] == "on") {
    digitalWrite(blue, HIGH);
    output["blue"] = analogRead(A2);
  } else {
    digitalWrite(blue, LOW);
    output["blue"] = analogRead(A2);
  }

 // create JSON object to send back data
 char jsonBuffer[256];
 serializeJson(output, jsonBuffer);
 Serial.println(jsonBuffer);
 }
}
```

Subject: Assignment 2 – Ultrasonic Sensing

To: Christopher Peters, PhD.
Teaching Professor, Electrical and Computer Engineering Department
Drexel University

From: Sofia D'Alessandro
Undergraduate, College of Engineering
Drexel University

Purpose

The purpose of this memo is to demonstrate ultrasonic sensing using serial communication. Ultrasonic sensing uses sound waves to measure distance. The ultrasonic sensor emits an ultrasonic pulse, or sound wave, at an object. The sound wave hits the object and bounces back, and then the reflected pulse is received by the sensor. The actual distance between the sensor and the object can then be calculated by taking half of the time it took to send and receive the pulse and multiply it by the speed of sound.

Methodology/Approach

In conducting the lab, the first step was to physically build the circuit using a solderless breadboard, the Arduino Uno R4 WiFi, the HC-SR04 ultrasonic sensor, and jumper wires. The ultrasonic sensor was placed on the breadboard. The power rail of the breadboard was connected to the 5V pin on the Arduino and then the ultrasonic sensor's Vcc pin was connected to the power rail. The ground rail was connected to the ground pin on the Arduino and then the ultrasonic sensor's ground pin was connected to the ground rail. The ultrasonic sensor's trig pin was connected to pin 8 on the Arduino and the ultrasonic sensor's echo pin was connected to pin 9 on the Arduino.

After the circuit was built, the next step was to begin working in the two applications that would be communicating with each other, Python and Arduino IDE. The programming in the IDE was conducted first. With the ultrasonic sensor's trig and echo pins connected to their respective pins on the Arduino, the pins could be initialized in the IDE. Communication with the serial port was also initialized and the necessary variables were initialized as well, including the JSON formatted object needed to send the data to Python for processing. A for loop was created to run from a distance of 4 centimeters to a distance of 30 centimeters in 1-centimeter increments. Inside the for loop was another for loop that ran 100 times. Inside the for loop, a 10-microsecond pulse was sent to the trig pin, and the ultrasonic sensor emitted a pulse toward the object. The echo pin was set to high when the pulse was sent and then was set to low when the pulse was received. The time that the echo pin was set to high was then multiplied by ½ the speed of sound to receive the distance. This was done 100 times until the for loop was finished. Each reading was added to a JSON array. Each time the loop finished, the JSON object was sent to Python. Then the previous JSON array was cleared, and the new array was initialized to prevent memory issues.

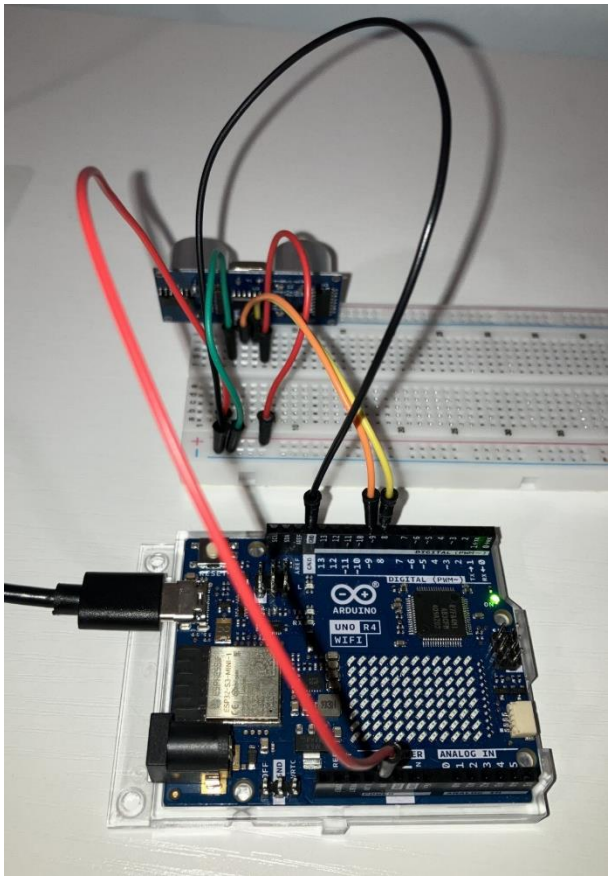
A connection to the serial port was established on the Python side. A function to read the data sent from the Arduino was created. In the main part of the Python program, the necessary variables were initialized. A while loop was created and, inside of the loop, the data from the Arduino was read using the function. If there was a response from the Arduino, the JSON object was converted into an object that Python could understand. The data could then be processed. The readings at each distance were printed, and the averages and standard deviations for these readings were calculated and added to their own lists. Outside of the while loop and when the program was killed and all of the data was processed, the lists could be printed, the plots could be created, and the connection to the serial port could be closed.

With the construction of the circuit and the programming finished, testing could finally be commenced. The Arduino was connected to the computer using the USB connection, or Serial port. The ultrasonic sensor was faced towards a wall 4 centimeters away with a tape measure set up next to it. The IDE code was uploaded to the Arduino and then the Python code was run. When the ultrasonic sensor was finished taking and sending the readings, and Python was done processing the data, a delay began. The ultrasonic sensor was then moved back another centimeter and, when the delay was finished, the ultrasonic sensor began taking readings again and sending them to Python. This process continued until the ultrasonic sensor finished taking readings at 30 centimeters away from the wall. When it was finished, the program was killed, and the final results and plots were outputted.

Results

The project was successful. The ultrasonic sensor was able to emit and receive pulses. The Arduino was able to read, calculate, and send the distances to Python for further processing. The output of the Python code looked reasonable. Overall, the readings at each distance varied but the averages were close to the actual distance. As the ultrasonic sensor got further away from the object, though, the averages showed as less than the actual distance. A factor for this could have been that the sensor was not exactly at the distance it should have been due to the not having the proper equipment and only using a tape measure to gauge the distances. The graph of averages versus distance looked good, though, as it increased linearly. The graph of standard deviations versus distance did not look as good. The standard deviations at each distance fluctuated, going anywhere from almost 0 to about 0.7. While there was not much of a pattern, it can clearly be seen that there was more fluctuation at the shorter distances as the standard deviation had the highest spikes there. This can be expected due to the shorter amount of time it would take to send and receive the pulses. The fact that the sensor was so close to the object could have resulted in some noise that could have affected the readings as well.

Picture of hardware:



Sample of output of Python code:

Connected to COM3 at 9600 baud
Received:
{"distance":4,"readings":[3.213,3.264,3.264,3.264,3.264,3.264,3.264,3.264,3.264,3.247,3.264,4.539,4.539,4.539,4.539,4.539,4.539,4.539,4.539,4.539,4.539,4.539,3.264,4.539,4.539,4.539,4.539,3.264,4.539,3.264,3.264,3.264,3.247,3.264,3.247,3.264]}
Distance = 4 cm, 100 readings
Average: 3.6709799999999984
Standard Deviation: 0.5954846426231327
...
Serial port closed.
Averages: [3.6709799999999984, 5.113089999999992, 5.647739999999995, 7.422539999999993, 7.987619999999996, 8.931120000000002, 10.42168000000002, 10.693000000000007, 11.779979999999975, 13.754529999999994, 14.193639999999998, 14.811420000000016, 15.845700000000024, 16.129430000000035, 16.558339999999983, 17.51322999999998, 19.243829999999974, 20.19531999999997, 21.40435999999998, 21.77734000000003, 22.940310000000025, 23.51320999999999, 25.01039999999999, 25.788150000000023, 26.984949999999984, 28.646530000000045, 28.935700000000043]

Standard Deviations: [0.5954846426231327, 0.6868961652972014, 0.3412775298785433, 0.008251569547668055, 0.6517766301425661, 0.5220938858098221, 0.003331306050185364, 0.5440000000000005, 0.004037276309593185, 0.004865089927226269, 0.41650124897771906, 0.6138041573661748, 0.1467229020977982, 0.1515632709464927, 0.388843033112334, 0.4181771599453989, 0.002939574799184305, 0.003331306050185713, 0.224550373858518, 0.0023799999999999247, 0.4882360637027953, 0.5152972015254879, 0.4570566266886408, 0.33493343741704856, 0.11359193413266618, 0.15702875246272538, 0.005638262143604001]
Distances: [4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30]

Appendix

Python code:

```
import serial, json, time, matplotlib.pyplot as plt, numpy as np

# Configure the serial port
port = 'COM3'
baudrate = 9600
try:
    ser = serial.Serial(port, baudrate, timeout=1)
    print(f"Connected to {port} at {baudrate} baud")
except serial.SerialException as e:
    print(f"Error opening serial port: {e}")
    exit()

# Function to read data
def read_data():
    if ser.in_waiting > 0:
        data = ser.readline().decode('utf-8').rstrip()
        print(f"Received: {data}")
        return data
    return None

# Main
try:
    # Initialize lists
    data = []
    averages = []
    std_devs = []
    distances = []
    while True:
        response = read_data() # Read data
        # Initialize variables
        sum = 0
        avg = 0
```

```
        std_dev = 0
        if response:
            data_object = json.loads(response) # Convert JSON object to Python object
            data.append(data_object) # Append data to list
            # Print the distance and number of readings
            print(f"Distance = {data_object.get("distance")} cm, {len(data_object.get("readings",
[]))} readings")
            # Calculate and print average
            for reading in data_object.get("readings", []):
                sum += reading
            avg = sum / len(data_object.get("readings", []))
            print(f"Average: {avg}")
            averages.append(avg) # Add average to list
            # Calculate and print standard deviation
            std_dev = float(np.std(data_object.get("readings", [])))
            print(f"Standard Deviation: {std_dev}")
            std_devs.append(std_dev) # Add standard deviation to list
            distances.append(data_object.get("distance")) # Add distance to list

        time.sleep(10) # Wait for 10 seconds

except KeyboardInterrupt:
    # Close the serial port
    ser.close()
    print("Serial port closed.")

    # Print the results
    print(f"Averages: {averages}")
    print(f"Standard Deviations: {std_devs}")
    print(f"Distances: {distances}")

    # Plot the averages
    plt.figure()
    plt.plot(distances, averages)
    plt.xlabel("Distance (cm)")
    plt.ylabel("Average of Readings (cm)")
    plt.xlim(0, 35)
    plt.ylim(0, 35)
    plt.title("Average of Readings vs Distance")
    plt.grid(True, which="major", axis="both", color="black", linestyle="-", linewidth=1,
alpha=1)
    plt.minorticks_on()
    plt.grid(True, which="minor", axis="both", color="black", linestyle="-", linewidth=0.3,
alpha=0.5)
    plt.show()
```

```
    # Plot the standard deviations
    plt.figure()
    plt.plot(distances, std_devs)
    plt.xlabel("Distance (cm)")
    plt.ylabel("Standard Deviation of Readings (cm)")
    plt.xlim(0, 35)
    plt.ylim(0, 1)
    plt.title("Standard Deviation of Readings vs Distance")
    plt.grid(True, which="major", axis="both", color="black", linestyle="-", linewidth=1,
alpha=1)
    plt.minorticks_on()
    plt.grid(True, which="minor", axis="both", color="black", linestyle="-", linewidth=0.3,
alpha=0.5)
    plt.show()
```

Arduino code:

```
#include <ArduinoJson.h>

#define TRIG_PIN  8 // The Arduino UNO R4 pin connected to the ultrasonic sensor's TRIG pin
#define ECHO_PIN  9 // The Arduino UNO R4 pin connected to the ultrasonic sensor's ECHO
pin

// initialize variables
float duration_us, distance_cm;
JsonDocument doc;
JsonArray readings;

void setup() {
  // begin serial port
  Serial.begin(9600);

  // configure the trigger pin to output mode
  pinMode(TRIG_PIN, OUTPUT);
  // configure the echo pin to input mode
  pinMode(ECHO_PIN, INPUT);
}

void loop() {
  for (int i = 4; i <= 30; i++) { // loop for each distance increment
    doc.clear(); // clear the previous readings array
    doc["distance"] = i;
    readings = doc.createNestedArray("readings"); // create a new JSON array for readings
    for (int j = 0; j < 100; j++) { // loop for each reading taken
      // generate 10-microsecond pulse to TRIG pin
      digitalWrite(TRIG_PIN, HIGH);
```

```
    delayMicroseconds(10);
    digitalWrite(TRIG_PIN, LOW);

    duration_us = pulseIn(ECHO_PIN, HIGH); // Duration of pulse from ECHO pin
    distance_cm = 0.017 * duration_us; // calculate the distance

    readings.add(distance_cm); // store each reading in JSON array
    delay(100);
   }
   serializeJson(doc, Serial);
   Serial.println();
   delay(10000);
  }
}
```

Output of Python code:

```
Connected to COM3 at 9600 baud
Received:
{"distance":4,"readings":[3.213,3.264,3.264,3.264,3.264,3.264,3.264,3.264,3.264,3.247,3.264,3.
264,3.264,3.264,3.264,3.264,3.264,3.264,3.264,3.264,3.264,3.264,3.264,3.264,3.264,3.264,3.264
,3.264,3.264,3.264,3.264,3.264,3.264,3.264,3.264,3.264,3.264,3.264,4.539,4.539,4.539,4.539,4.5
39,4.539,4.539,4.539,4.539,3.264,4.539,4.539,4.539,4.539,3.264,4.539,4.539,4.539,4.539,4.539,
4.539,4.539,4.539,4.539,4.539,4.539,4.539,4.539,4.539,4.539,4.539,4.539,4.539,4.539,3.264,3.2
64,3.264,3.247,3.264,3.247,3.264,3.264,3.264,3.264,3.264,3.264,3.264,3.264,3.264,3.264,3.264,
3.264,3.264,3.264,3.264,3.264,3.264,3.264,3.264,3.264,3.264,3.264]}
Distance = 4 cm, 100 readings
Average: 3.6709799999999984
Standard Deviation: 0.5954846426231327
Received:
{"distance":5,"readings":[3.264,3.264,3.264,3.264,3.264,3.247,3.264,3.893,3.57,4.216,4.216,4.2
16,4.216,4.216,5.168,4.862,4.539,4.539,4.539,4.539,4.539,4.539,4.539,5.508,5.491,5.508,5.508,
5.508,5.508,5.508,5.508,5.508,5.491,4.539,4.539,4.539,4.539,4.539,4.539,5.508,5.508,5.491,5.5
08,5.508,5.508,5.508,5.508,5.491,5.491,5.508,5.508,5.508,5.508,5.491,5.508,5.508,5.508,5.508,
5.508,5.508,5.508,5.508,5.491,5.508,5.508,5.508,5.508,5.508,5.508,5.491,5.508,5.508,5.508,5.5
08,5.508,5.508,5.508,5.491,5.491,5.508,5.508,5.508,5.508,5.508,5.508,5.508,5.508,5.508,5.508,
5.508,5.508,5.508,5.508,5.508,5.508,5.508,5.508,5.508,5.508,5.508]}
Distance = 5 cm, 100 readings
Average: 5.113089999999992
Standard Deviation: 0.6868961652972014
Received:
{"distance":6,"readings":[5.508,5.508,5.508,5.491,5.508,5.491,5.508,5.508,5.508,5.508,5.508,5.
508,5.508,5.508,5.508,5.508,5.508,5.508,5.508,5.491,5.508,5.508,5.491,5.508,5.508,5.508,5.508,5.508
,5.508,5.508,5.508,5.508,5.508,5.508,5.508,5.508,5.508,5.508,5.508,5.491,5.491,5.508,5.491,5.4
91,5.508,5.491,5.508,5.508,5.508,5.508,5.508,5.491,5.491,5.508,5.508,5.491,5.508,5.508,5.508,
5.508,5.508,5.491,5.508,5.508,5.508,5.508,5.508,5.508,5.508,5.508,5.508,5.508,5.508,5.491,5.4
```

91,6.46,5.508,6.46,6.46,6.46,6.46,6.46,6.46,6.46,6.46,6.46,6.46,6.46,6.46,5.491,5.508,6.46,5.508
,5.508,5.491,6.46,5.508,5.508,5.508,5.508,5.491]}
Distance = 6 cm, 100 readings
Average: 5.647739999999995
Standard Deviation: 0.3412775298785433
Received:
{"distance":7,"readings":[7.429,7.429,7.412,7.429,7.429,7.412,7.429,7.412,7.429,7.429,7.412,7.
412,7.429,7.429,7.412,7.429,7.429,7.429,7.429,7.412,7.429,7.429,7.429,7.412,7.429,7.429,7.429
,7.429,7.429,7.429,7.429,7.429,7.412,7.412,7.429,7.429,7.429,7.412,7.412,7.412,7.429,7.412,7.4
12,7.429,7.412,7.429,7.429,7.429,7.412,7.412,7.412,7.412,7.412,7.429,7.429,7.429,7.429,7.412,
7.429,7.429,7.429,7.412,7.429,7.429,7.429,7.412,7.412,7.412,7.412,7.429,7.412,7.412,7.412,7.429,7.4
29,7.429,7.429,7.412,7.429,7.429,7.429,7.429,7.412,7.429,7.429,7.412,7.412,7.429,7.412,7.429,
7.429,7.429,7.429,7.429,7.412,7.429,7.412,7.429,7.412,7.429,7.412]}
Distance = 7 cm, 100 readings
Average: 7.422539999999993
Standard Deviation: 0.008251569547668055
Received:
{"distance":8,"readings":[7.412,8.738,8.738,8.738,8.738,8.738,8.738,8.738,8.738,8.738,8.738,8.
738,8.738,8.738,8.738,8.738,8.738,8.738,8.738,8.738,8.738,8.738,7.429,8.738,8.738,8.738,7.412,7.412
,8.738,8.738,8.738,8.738,8.738,8.738,8.738,8.738,8.738,8.738,8.738,8.738,7.429,7.429,7.429,7.4
12,8.738,8.738,8.738,8.738,8.738,8.738,8.738,8.738,7.429,7.412,7.429,7.429,7.429,7.412,7.412,
7.429,7.412,7.412,7.412,7.412,7.429,7.412,7.429,7.429,7.412,7.429,7.429,7.412,7.429,7.412,7.4
29,7.412,7.429,7.429,7.412,7.429,7.429,7.412,7.429,7.429,7.412,7.429,7.429,7.429,7.429,7.429,
7.412,7.429,7.429,7.429,7.412,7.429,7.412,7.412,7.429,7.412,7.412]}
Distance = 8 cm, 100 readings
Average: 7.987619999999996
Standard Deviation: 0.6517766301425661
Received:
{"distance":9,"readings":[8.738,8.738,8.738,8.738,8.738,8.738,8.738,8.738,8.738,8.738,8.738,8.
738,8.738,8.738,8.738,8.738,8.738,8.738,8.738,8.738,8.738,8.738,8.738,8.738,8.738,8.738,8.738
,8.738,8.738,8.738,8.738,8.381,8.381,8.738,8.738,8.738,8.738,8.738,8.738,8.738,8.738,8.738,8.7
38,8.738,8.738,9.758,8.738,8.738,9.758,9.758,8.381,8.381,8.381,9.758,8.381,9.758,8.738,8.738,
9.758,9.758,8.738,8.738,9.758,9.758,9.758,9.758,8.738,9.758,9.758,8.381,8.381,8.381,9.758,8.3
81,9.758,8.381,9.758,8.381,9.741,9.758,9.758,9.758,8.381,9.758,9.758,8.381,8.381,8.381,9.758,
9.758,9.758,9.758,9.758,8.381,8.381,8.381,8.381,8.381,8.381,8.381]}
Distance = 9 cm, 100 readings
Average: 8.931120000000002
Standard Deviation: 0.5220938858098221
Received:
{"distance":10,"readings":[10.421,10.421,10.421,10.421,10.421,10.421,10.421,10.421,10.421,10
.421,10.421,10.421,10.421,10.421,10.421,10.421,10.421,10.421,10.421,10.421,10.421,10.421,10
.421,10.421,10.421,10.421,10.421,10.421,10.421,10.421,10.421,10.421,10.421,10.421,10.421,10
.421,10.421,10.421,10.421,10.421,10.438,10.421,10.421,10.421,10.438,10.421,10.421,10
.421,10.421,10.421,10.421,10.421,10.421,10.421,10.421,10.421,10.421,10.421,10.421,10.421,10
.421,10.421,10.421,10.421,10.421,10.421,10.421,10.421,10.421,10.421,10.421,10.421,10.421,10

.421,10.421,10.421,10.438,10.421,10.421,10.421,10.421,10.421,10.421,10.421,10.421,10.421,10
.421,10.421,10.421,10.421,10.421,10.421,10.421,10.421,10.421,10.438,10.421,10.421,10.421]}
Distance = 10 cm, 100 readings
Average: 10.42168000000002
Standard Deviation: 0.003331306050185364
Received:
{"distance":11,"readings":[10.421,10.421,10.421,10.421,10.421,10.421,10.421,10.421,10.421,10
.421,10.421,10.421,10.421,10.421,10.421,10.421,10.421,10.421,10.421,11.781,10.421,10.421,10
.421,10.421,10.421,10.421,10.421,10.421,11.781,11.781,10.421,10.421,10.421,10.421,10.421,10
.421,10.421,10.421,10.421,10.421,10.421,10.421,10.421,10.421,10.421,10.421,10.421,10.421,10
.421,10.421,10.421,10.421,10.421,11.781,10.421,10.421,10.421,10.421,10.421,10.421,10.421,10
.421,10.421,10.421,10.421,10.421,10.421,10.421,10.421,10.421,10.421,10.421,10.421,10.421,10
.421,10.421,11.781,10.421,10.421,10.421,10.421,10.421,11.781,11.781,11.781,11.781,11.781,11.
781,11.781,10.421,11.781,11.781,11.781,11.781,11.781,10.421,11.781,11.781,10.421,11.781]}
Distance = 11 cm, 100 readings
Average: 10.693000000000007
Standard Deviation: 0.5440000000000005
Received:
{"distance":12,"readings":[11.781,11.781,11.781,11.781,11.781,11.781,11.781,11.781,11.781,11.
781,11.781,11.781,11.781,11.781,11.781,11.781,11.781,11.781,11.781,11.781,11.781,11.781,11.
781,11.781,11.781,11.764,11.781,11.781,11.781,11.781,11.764,11.781,11.781,11.781,11.781,11.
781,11.781,11.781,11.781,11.781,11.781,11.781,11.781,11.764,11.781,11.781,11.781,11.781,11.
781,11.764,11.781,11.781,11.781,11.781,11.781,11.781,11.781,11.781,11.764,11.781,11.781,11.
781,11.781,11.781,11.781,11.781,11.781,11.781,11.781,11.781,11.781,11.781,11.781,11.781,11.
764,11.781,11.781,11.781,11.781,11.781,11.781,11.781,11.781,11.781,11.781,11.781,11.781,11.
781,11.781,11.781,11.781,11.781,11.781,11.781,11.781,11.781,11.781,11.781,11.781,11.781]}
Distance = 12 cm, 100 readings
Average: 11.779979999999975
Standard Deviation: 0.004037276309593185
Received:
{"distance":13,"readings":[13.753,13.753,13.753,13.753,13.753,13.753,13.753,13.753,13.753,13
.753,13.753,13.753,13.753,13.753,13.753,13.753,13.753,13.753,13.753,13.753,13.77,13.77,13.7
53,13.753,13.753,13.753,13.753,13.753,13.753,13.753,13.77,13.753,13.753,13.77,13.753,13.753
,13.753,13.753,13.753,13.77,13.753,13.753,13.753,13.753,13.753,13.753,13.753,13.753,13.753,
13.753,13.753,13.753,13.77,13.753,13.77,13.753,13.753,13.753,13.753,13.753,13.753,13.753,13
.753,13.753,13.753,13.753,13.753,13.753,13.753,13.753,13.753,13.753,13.753,13.753,13.753,13
.753,13.753,13.753,13.753,13.753,13.753,13.77,13.753,13.753,13.753,13.753,13.753,13.753,13.
753,13.753,13.753,13.753,13.753,13.753,13.753,13.753,13.753,13.753,13.753,13.77]}
Distance = 13 cm, 100 readings
Average: 13.754529999999994
Standard Deviation: 0.004865089927226269
Received:
{"distance":14,"readings":[13.753,13.753,13.753,13.413,13.413,13.413,13.413,13.413,13.413,13
.413,13.753,13.413,13.413,13.413,13.413,13.413,13.413,13.413,13.413,13.77,13.736,13.413,14.
093,13.413,14.433,14.093,15.096,14.773,14.756,14.756,14.093,14.093,14.093,14.093,13.753,13.
753,14.093,14.433,14.433,14.433,14.433,14.433,14.433,14.416,14.433,14.433,14.433,14.433,14.

433,14.093,14.433,14.433,14.433,14.433,14.433,14.433,14.433,14.433,14.433,14.433,14.433,14.
433,14.433,14.433,14.433,14.433,14.433,14.416,14.433,14.433,14.433,14.416,14.433,14.433,14.
433,14.433,14.433,14.433,14.416,14.433,14.433,14.433,14.416,14.433,14.433,14.433,14.433,14.
433,14.433,14.433,14.433,14.433,14.433,14.433,14.433,14.433,14.433,14.433,14.433,14.433]}
Distance = 14 cm, 100 readings
Average: 14.193639999999998
Standard Deviation: 0.41650124897771906
Received:
{"distance":15,"readings":[13.413,13.413,13.413,13.413,13.413,13.413,13.413,13.413,13.413,13
.413,13.413,13.43,13.413,13.413,14.433,14.433,14.773,14.093,14.773,14.433,14.433,14.433,14.
433,15.776,15.453,15.453,15.453,15.453,15.453,15.436,15.113,15.096,15.096,14.756,14.773,14.
756,14.773,14.773,14.773,14.773,14.756,14.773,15.113,15.096,15.113,15.096,15.096,15.096,15.
096,15.096,15.096,15.096,15.113,15.096,15.113,15.113,15.096,15.096,15.096,15.113,15.096,15.
096,15.113,15.113,15.113,15.113,15.096,15.113,15.113,15.113,15.113,15.113,15.113,15.096,15.
113,15.113,15.096,15.096,15.113,15.113,15.113,15.113,15.096,15.096,15.096,15.113,15.096,15.
113,15.113,15.113,15.113,15.096,15.096,15.113,15.096,15.096,15.096,15.453,15.096,15.113]}
Distance = 15 cm, 100 readings
Average: 14.811420000000016
Standard Deviation: 0.6138041573661748
Received:
{"distance":16,"readings":[15.453,16.065,15.759,15.759,15.759,15.759,15.759,15.759,15.759,15
.759,15.759,15.759,15.759,15.776,15.776,15.759,16.082,16.082,16.082,15.759,15.759,15.759,15
.759,15.759,15.759,15.759,15.759,15.776,15.759,15.759,15.759,15.759,15.759,15.759,15.759,15
.759,15.759,15.776,15.759,15.776,15.759,15.759,15.759,15.776,15.759,15.776,15.759,15.776,15
.776,15.776,15.776,15.776,15.776,15.759,15.759,16.082,15.759,15.759,15.759,15.759,15.759,16
.082,15.759,16.082,16.082,15.759,15.776,15.759,16.082,15.759,15.759,15.759,15.759,15.759,16
.082,16.082,15.759,15.759,16.082,16.082,16.082,16.082,16.082,15.776,15.759,15.776,15.759,15
.759,16.082,16.082,16.082,16.082,16.082,16.082,16.082,16.082,16.082,15.759,16.082]}
Distance = 16 cm, 100 readings
Average: 15.845700000000024
Standard Deviation: 0.1467229020977982
Received:
{"distance":17,"readings":[16.711,16.728,16.728,16.711,16.405,16.388,16.405,16.405,16.388,16
.711,16.082,16.082,16.082,16.082,16.082,16.082,16.082,16.082,16.082,16.082,16.082,16.082,16
.082,16.082,16.082,16.082,16.082,16.082,16.082,16.082,16.082,16.082,16.082,16.082,16.082,16
.082,16.082,16.082,16.082,16.082,16.082,16.082,16.082,16.082,16.082,16.082,16.082,16.082,16
.082,16.082,16.082,16.082,16.082,16.082,16.082,16.082,16.082,16.082,16.082,16.082,16.082,16
.082,16.082,16.082,16.065,16.082,16.082,16.082,16.082,16.082,16.082,16.082,16.082,16.082,16
.082,16.082,16.082,16.082,16.082,16.082,16.082,16.082,16.082,16.082,16.082,16.082,16.082,16
.082,16.082,16.082,16.082,16.082,16.082,16.082,16.082,16.082,16.082,16.082,16.082,16.082]}
Distance = 17 cm, 100 readings
Average: 16.129430000000035
Standard Deviation: 0.1515632709464927
Received:
{"distance":18,"readings":[17.357,17.34,17.34,17.357,17.34,17.34,16.405,16.388,16.388,16.405,
16.405,17.663,17.663,17.663,17.663,16.388,17.34,17.34,17.357,16.405,16.405,16.405,16.405,16

.388,16.405,16.388,16.405,16.388,16.388,16.388,16.388,16.405,16.388,16.388,16.388,16.405,16
.405,16.388,16.388,16.405,16.405,16.405,16.405,16.388,17.663,16.388,16.405,16.388,16.388,16
.405,16.388,16.405,16.388,17.663,16.405,16.388,16.388,16.405,16.388,16.388,16.388,16.405,16
.388,16.405,16.405,16.405,16.388,16.388,16.388,16.405,16.405,16.388,16.405,16.405,16.405,16
.388,16.405,16.388,16.388,16.388,16.405,16.405,16.405,16.405,16.405,16.405,16.388,16.405,16
.405,16.388,16.405,16.405,16.405,16.405,16.405,16.388,16.388,16.405,16.388,16.388]}
Distance = 18 cm, 100 readings
Average: 16.558339999999983
Standard Deviation: 0.388843033112334
Received:
{"distance":19,"readings":[17.663,17.34,17.357,17.34,17.357,17.357,17.357,17.357,17.357,17.3
4,17.357,17.357,17.34,18.615,18.615,18.615,18.615,18.615,18.615,18.615,18.615,18.598,18.615
,18.615,18.615,17.34,17.34,17.34,17.34,17.357,17.34,17.34,17.34,17.357,17.357,17.357,17.34,1
7.357,17.34,17.34,17.34,17.357,17.34,17.34,17.34,17.357,17.357,17.357,17.34,17.34,17.357,17.
357,17.34,17.357,17.357,17.34,17.357,17.34,17.34,17.357,17.357,17.34,17.34,17.34,18.292,17.3
57,17.357,17.34,17.34,17.357,17.357,17.357,17.357,17.357,17.34,17.34,17.34,17.34,17.34,17.35
7,17.357,17.357,17.357,17.357,17.357,17.357,17.357,17.34,17.34,17.34,17.357,17.34,17.34,17.3
57,17.357,17.34,17.357,17.34,17.357,17.357]}
Distance = 19 cm, 100 readings
Average: 17.51322999999998
Standard Deviation: 0.4181771599453989
Received:
{"distance":20,"readings":[19.244,19.244,19.244,19.244,19.244,19.244,19.244,19.244,19.244,19
.244,19.244,19.244,19.244,19.261,19.244,19.244,19.244,19.244,19.244,19.244,19.244,19.244,19
.244,19.244,19.244,19.244,19.244,19.244,19.244,19.244,19.244,19.244,19.244,19.244,19.244,19
.244,19.244,19.244,19.244,19.244,19.244,19.244,19.244,19.244,19.244,19.244,19.244,19.244,19
.244,19.244,19.244,19.244,19.244,19.244,19.244,19.244,19.227,19.244,19.244,19.244,19.244,19
.244,19.244,19.244,19.244,19.244,19.244,19.244,19.244,19.244,19.244,19.244,19.244,19.244,19
.244,19.244,19.244,19.244,19.244,19.244,19.244,19.244,19.244,19.244,19.244,19.244,19.244,19
.244,19.227,19.244,19.244,19.244,19.244,19.244,19.244,19.244,19.244,19.244,19.244,19.244]}
Distance = 20 cm, 100 readings
Average: 19.243829999999974
Standard Deviation: 0.002939574799184305
Received:
{"distance":21,"readings":[20.196,20.196,20.196,20.196,20.196,20.196,20.196,20.196,20.196,20
.196,20.196,20.196,20.196,20.196,20.196,20.196,20.196,20.196,20.196,20.196,20.196,20.196,20
.196,20.196,20.196,20.179,20.196,20.196,20.196,20.196,20.196,20.196,20.196,20.196,20.196,20
.196,20.196,20.196,20.196,20.196,20.196,20.196,20.196,20.196,20.196,20.196,20.196,20.196,20
.196,20.196,20.196,20.196,20.196,20.196,20.196,20.196,20.196,20.179,20.196,20.196,20.196,20
.196,20.196,20.196,20.196,20.196,20.179,20.196,20.196,20.196,20.196,20.196,20.196,20.196,20
.196,20.196,20.196,20.196,20.196,20.196,20.196,20.196,20.196,20.196,20.196,20.196,20.196,20
.196,20.196,20.196,20.196,20.196,20.196,20.196,20.196,20.196,20.179,20.196,20.196]}
Distance = 21 cm, 100 readings
Average: 20.19531999999997
Standard Deviation: 0.003331306050185713

Received:
{"distance":22,"readings":[21.454,21.454,21.471,21.454,21.454,21.454,21.471,21.471,21.454,21
.454,21.454,21.454,21.454,21.471,21.454,21.454,21.454,21.471,21.454,21.454,21.454,21.471,21
.471,21.454,21.471,21.471,21.454,21.471,21.471,21.454,21.454,21.454,21.471,21.454,21.454,21
.454,21.454,21.471,21.471,21.454,21.454,21.454,21.454,21.471,21.471,21.454,21.471,21.471,21
.454,21.471,21.454,21.471,21.454,21.454,21.471,21.454,21.471,21.471,21.471,21.454,21.471,21
.454,21.454,21.454,21.471,21.471,21.471,21.454,21.454,21.471,21.454,21.454,21.471,21.471,21
.454,21.471,21.454,21.471,21.454,21.454,21.471,21.471,21.454,21.454,21.454,21.454,21.471,21
.471,21.454,21.454,21.454,21.471,21.471,20.519,20.502,20.519,20.519,20.519,21.454,20.519]}
Distance = 22 cm, 100 readings
Average: 21.40435999999998
Standard Deviation: 0.224550373858518
Received:
{"distance":23,"readings":[21.777,21.777,21.777,21.777,21.777,21.777,21.777,21.777,21.777,21
.777,21.777,21.777,21.777,21.777,21.777,21.777,21.777,21.777,21.777,21.794,21.777,21.777,21
.777,21.777,21.794,21.777,21.777,21.777,21.777,21.777,21.777,21.777,21.777,21.777,21.777,21
.777,21.777,21.777,21.777,21.777,21.777,21.777,21.777,21.777,21.777,21.777,21.777,21.777,21
.777,21.777,21.777,21.777,21.777,21.777,21.777,21.777,21.777,21.777,21.777,21.777,21.777,21
.777,21.777,21.777,21.777,21.777,21.777,21.777,21.777,21.777,21.777,21.777,21.777,21.777,21
.777,21.777,21.777,21.777,21.777,21.777,21.777,21.777,21.777,21.777,21.777,21.777,21.777,21
.777,21.777,21.777,21.777,21.777,21.777,21.777,21.777,21.777,21.777,21.777,21.777,21.777]}
Distance = 23 cm, 100 readings
Average: 21.77734000000003
Standard Deviation: 0.0023799999999999247
Received:
{"distance":24,"readings":[22.661,21.777,21.777,22.661,22.083,22.083,22.083,22.1,22.1,22.083,
22.1,22.083,22.661,22.083,22.083,22.661,21.777,21.777,22.372,22.389,22.083,23.528,23.239,22
.95,22.95,23.511,23.511,23.511,23.528,23.528,23.528,22.661,22.661,22.661,22.661,23.239,23.2
22,22.661,22.661,22.661,22.661,22.661,22.661,22.661,22.661,22.661,22.644,22.661,22.661,24.0
89,23.239,23.222,23.222,23.239,23.222,23.239,23.222,23.239,23.239,23.239,23.222,23.222,23.2
39,23.222,23.239,23.239,23.239,23.239,23.239,23.239,23.239,23.239,23.239,23.239,23.222,23.2
39,23.239,23.222,23.239,23.239,23.239,23.222,23.239,23.239,23.222,23.239,23.239,23.239,23.2
39,23.222,23.239,23.222,23.239,23.222,23.239,23.239,23.239,23.222,23.222,23.222]}
Distance = 24 cm, 100 readings
Average: 22.940310000000025
Standard Deviation: 0.4882360637027953
Received:
{"distance":25,"readings":[23.222,23.222,23.239,23.222,23.239,22.372,23.222,23.222,23.239,23
.239,23.239,23.222,23.239,23.222,23.239,23.239,23.239,23.239,23.222,23.239,23.222,23.222,23
.222,23.222,23.222,23.239,23.239,23.239,23.239,23.817,23.239,23.239,23.8,23.239,23.239,23.2
22,22.661,22.661,22.661,22.661,22.661,22.661,22.661,22.661,22.661,22.661,22.661,24.089,23.2
22,23.239,23.239,23.239,22.95,22.95,22.95,23.511,23.511,24.089,23.222,23.239,23.817,23.817,
23.8,24.378,23.511,23.528,24.089,24.089,24.089,24.089,24.089,24.089,24.089,24.089,24.106,24
.089,24.089,24.089,24.089,24.106,24.089,24.106,24.089,24.106,24.089,24.089,24.089,24.089,24
.089,24.089,24.089,24.089,24.089,24.089,24.089,24.089,24.106,24.089,24.089,24.089]}
Distance = 25 cm, 100 readings

Average: 23.51320999999999
Standard Deviation: 0.5152972015254879
Received:
{"distance":26,"readings":[24.089,24.089,23.817,24.378,24.089,24.667,25.806,24.956,24.956,24
.667,24.667,25.245,25.245,24.956,24.956,24.956,25.823,25.806,25.245,25.245,25.245,25.245,25
.245,25.245,26.112,24.667,24.65,24.667,26.112,25.245,24.667,24.684,24.667,24.667,24.667,24.
667,24.684,24.667,24.667,24.667,24.667,24.684,25.534,25.534,24.956,24.956,24.956,24.956,24.
956,24.956,24.956,24.956,24.956,25.517,24.667,24.667,24.684,24.667,24.667,24.667,24.667,24.
684,24.667,24.667,24.684,25.534,24.667,24.667,24.667,24.667,24.667,24.667,24.667,24.667,24.
65,24.684,24.684,25.534,24.667,25.517,24.65,25.534,25.534,25.517,25.534,25.534,25.534,25.51
7,25.534,25.517,25.534,25.534,25.534,25.534,25.517,25.534,25.517,25.534,25.534,25.534]}
Distance = 26 cm, 100 readings
Average: 25.01039999999999
Standard Deviation: 0.4570566266886408
Received:
{"distance":27,"readings":[25.806,25.823,25.823,25.823,25.823,25.823,25.823,25.806,26.112,26
.112,26.112,26.112,26.112,26.095,26.095,26.673,26.69,26.095,26.112,26.095,26.095,26.112,26.
095,26.112,26.095,26.112,26.112,26.112,26.112,26.095,26.112,26.095,26.112,26.095,26.112,26.
112,26.095,26.112,26.095,26.962,25.534,25.534,25.517,25.517,25.517,25.517,25.517,25.517,25.
517,25.517,25.517,25.517,25.517,25.534,25.517,25.517,25.534,25.534,25.517,25.517,25.534,25.
517,25.517,25.517,26.384,26.384,26.384,26.384,25.517,25.517,25.534,25.534,25.517,25.517,25.
534,25.534,25.534,25.534,25.517,25.517,25.517,25.517,25.534,25.534,25.534,25.534,25.534,25.
534,25.517,25.534,25.517,25.534,25.534,25.517,25.534,25.517,25.534,25.534,25.517,25.517]}
Distance = 27 cm, 100 readings
Average: 25.788150000000023
Standard Deviation: 0.33493343741704856
Received:
{"distance":28,"readings":[26.962,26.962,26.962,26.962,27.54,27.54,27.54,27.54,26.962,26.962,
26.962,26.962,26.962,26.962,26.962,26.979,26.945,26.962,26.962,26.945,26.962,26.962,26.962,
26.945,26.962,26.962,26.945,26.962,26.962,26.945,26.945,26.962,26.962,26.962,26.979,26.962,
26.962,26.979,26.962,26.962,26.962,26.962,26.962,26.979,26.962,26.962,26.962,26.962,26.962,
26.962,26.962,26.962,26.962,26.962,26.962,26.962,26.962,26.962,26.945,26.962,26.962,26.962,
26.962,26.945,26.962,26.945,26.962,26.962,26.962,26.962,26.962,26.979,26.962,26.962,26.945,
26.945,26.962,26.962,26.962,26.962,26.945,26.962,26.962,26.962,26.962,26.962,26.962,26.979,
26.979,26.979,26.962,26.979,26.979,26.962,26.979,26.962,26.962,26.962,26.962,26.962]}
Distance = 28 cm, 100 readings
Average: 26.984949999999984
Standard Deviation: 0.11359193413266618
Received:
{"distance":29,"readings":[27.812,28.679,28.679,28.679,28.679,28.679,28.679,28.679,28.679,28
.679,28.679,28.679,28.679,28.679,28.679,27.812,28.679,28.679,28.679,28.679,28.679,28.662,28
.679,28.679,28.679,28.662,28.679,28.679,28.679,28.679,28.679,28.679,28.679,28.679,28.679,28
.679,28.679,28.662,28.679,28.679,28.679,28.679,28.679,28.679,28.662,28.679,28.679,28.679,28
.679,28.679,28.679,28.679,28.679,28.679,28.679,28.679,28.679,28.679,28.679,28.679,28.118,28
.679,28.679,28.679,28.679,28.679,28.679,28.679,28.679,28.679,28.679,28.679,28.679,28.679,28

.662,28.679,28.679,28.679,28.679,28.679,28.679,28.679,28.679,28.679,28.679,28.679,28.679,28
.679,28.679,28.679,28.679,28.679,28.679,28.679,27.812,28.679,28.679,28.679,28.679,28.679]}
Distance = 29 cm, 100 readings
Average: 28.646530000000045
Standard Deviation: 0.15702875246272538
Received:
{"distance":30,"readings":[28.934,28.934,28.951,28.934,28.934,28.934,28.951,28.934,28.934,28
.934,28.934,28.934,28.934,28.934,28.934,28.934,28.917,28.934,28.934,28.934,28.934,28.934,28
.934,28.934,28.951,28.934,28.934,28.934,28.934,28.934,28.934,28.934,28.934,28.934,28.934,28
.934,28.934,28.934,28.934,28.934,28.934,28.934,28.934,28.951,28.951,28.934,28.934,28
.934,28.934,28.934,28.934,28.934,28.934,28.934,28.934,28.934,28.934,28.934,28.934,28.934,28
.934,28.934,28.934,28.934,28.934,28.951,28.934,28.951,28.951,28.934,28.951,28.934,28.934,28
.934,28.934,28.934,28.934,28.934,28.934,28.934,28.934,28.951,28.934,28.934,28.934,28.934,28
.934,28.934,28.934,28.951,28.934,28.934,28.934,28.934,28.934,28.934,28.934,28.934,28.934]}
Distance = 30 cm, 100 readings
Average: 28.935700000000043
Standard Deviation: 0.005638262143604001
Serial port closed.
Averages: [3.6709799999999984, 5.113089999999992, 5.647739999999995,
7.422539999999993, 7.987619999999996, 8.931120000000002, 10.42168000000002,
10.693000000000007, 11.779979999999975, 13.754529999999994, 14.193639999999998,
14.811420000000016, 15.845700000000024, 16.129430000000035, 16.558339999999983,
17.51322999999998, 19.243829999999974, 20.19531999999997, 21.40435999999998,
21.77734000000003, 22.940310000000025, 23.51320999999999, 25.01039999999999,
25.788150000000023, 26.984949999999984, 28.646530000000045, 28.935700000000043]
Standard Deviations: [0.5954846426231327, 0.6868961652972014, 0.3412775298785433,
0.008251569547668055, 0.6517766301425661, 0.5220938858098221, 0.003331306050185364,
0.5440000000000005, 0.004037276309593185, 0.004865089927226269,
0.41650124897771906, 0.6138041573661748, 0.1467229020977982, 0.1515632709464927,
0.388843033112334, 0.4181771599453989, 0.002939574799184305, 0.003331306050185713,
0.224550373858518, 0.0023799999999999247, 0.4882360637027953, 0.5152972015254879,
0.4570566266886406, 0.33493343741704856, 0.11359193413266618, 0.15702875246272538,
0.005638262143604001]
Distances: [4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28,
29, 30]

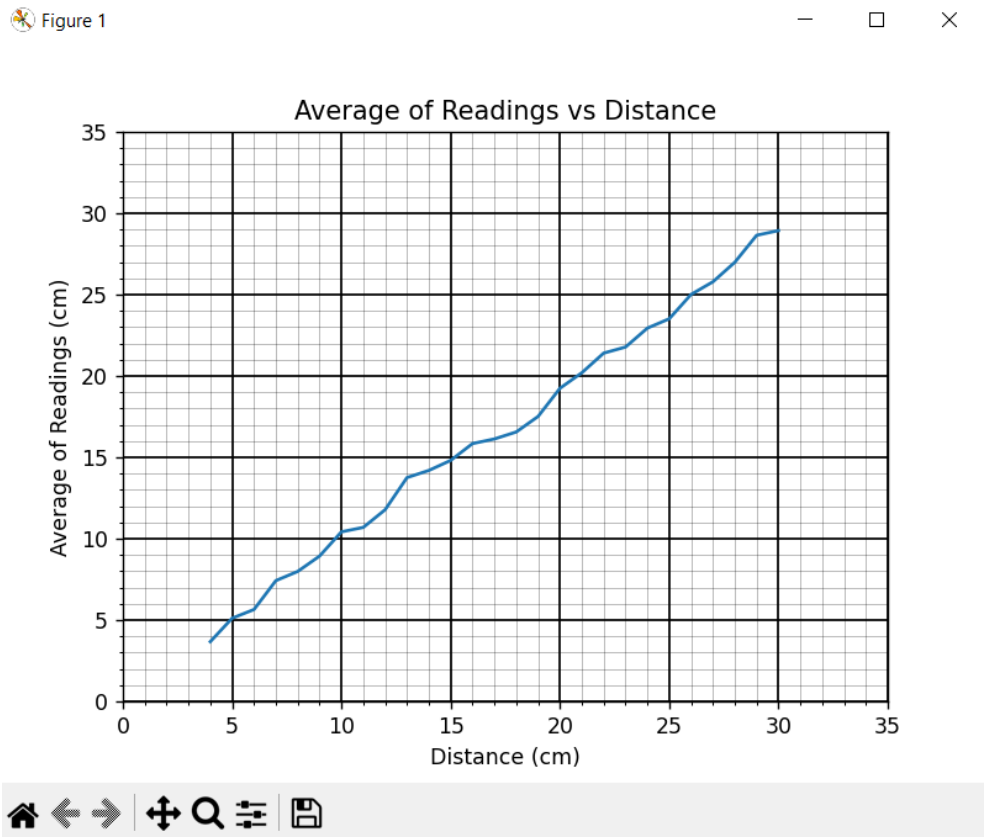


Figure 1
Average of Readings vs Distance
Average of Readings (cm)
Distance (cm)
0
5
10
15
20
25
30
35

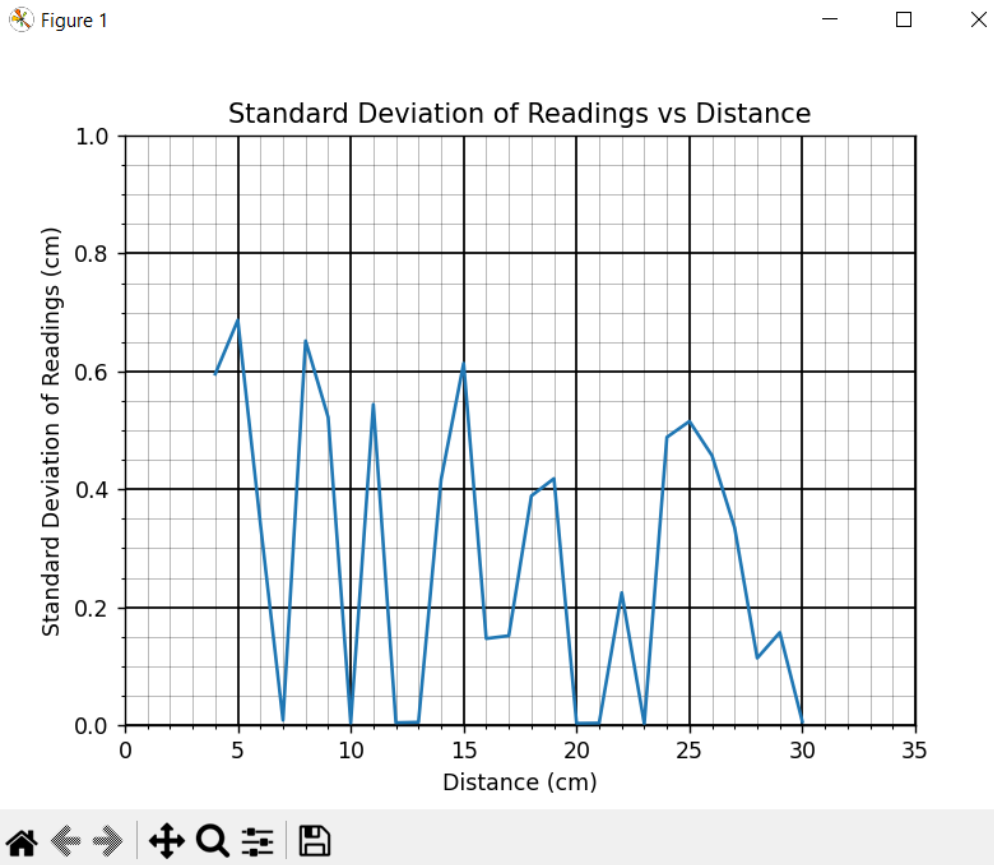


Figure 1
Standard Deviation of Readings vs Distance
Standard Deviation of Readings (cm)
Distance (cm)
0.0
0.2
0.4
0.6
0.8
1.0
0
5
10
15
20
25
30
35

Subject: Assignment 3

To: Christopher Peters, PhD.
Teaching Professor, Electrical and Computer Engineering Department
Drexel University

From: Sofia D'Alessandro
Undergraduate, College of Engineering
Drexel University

Purpose

The purpose of this memo is to demonstrate all of the topics covered thus far. Midway through the course, using the serial port to send and receive data as JSON strings has been done in a variety of ways. This project is the culmination of all of them. However, instead of sending and receiving data to and from Python, Node-Red was used in its place. The Arduino is able to communicate with Node-Red through communication with the serial port. Node-Red is used to read data sent from Arduino. It is able to process the data and format it for easier and more functional interpretation. In addition to being able to send data to Node-Red, Arduino is also able to receive data from Node-Red.

Methodology/Approach

In conducting the lab, the first step was to physically build the circuit using a solderless breadboard, the Arduino Uno R4 WiFi, the HC-SR04 ultrasonic sensor, the GY-87 sensor, one red LED, one blue LED, two 1000-ohm resistors, and jumper wires. The ultrasonic sensor was placed on the breadboard. The power rail of the breadboard was connected to the 5V pin on the Arduino and then the ultrasonic sensor's Vcc pin and the GY-87 sensor's Vcc_in pin were connected to the power rail. The ground rail was connected to a ground pin on the Arduino and then the ultrasonic sensor's ground pin was connected to the ground rail. The ultrasonic sensor's trig pin was connected to pin 12 on the Arduino and the ultrasonic sensor's echo pin was connected to pin 13 on the Arduino. The GY-87 sensor's ground pin was connected to a ground pin on the Arduino and the GY-87 sensor's 3.3V pin was connected to the 3.3V pin on the Arduino. The GY-87 sensor's SCL pin was connected to the SCL pin on the Arduino and the GY-87 sensor's SDA pin was connected to the SDA pin on the Arduino. The red and blue LEDS were then placed on the breadboard. Their cathodes were connected to the other ground rail while their anodes were connected to the resistors, which were then connected to pin 6 and pin 5, respectively. Finally, both ground rails were connected.

With the circuit connected, programming in the IDE could begin. First, all of the required libraries and necessary variables were initialized. Communication with the serial port was also initialized. Functions were written to initialize the primary sensors in the GY-87 sensor, the MPU 6050 accelerometer and gyroscope, the QMC5883L digital compass, which had to be calibrated using the compass_calibration.ino code, and the BMP180 pressure and temperature sensor. Functions were also written to gather the data from each sensor and add it to the JSON object that would send the data to Node-Red. Inside of the loop, the ultrasonic sensor was called to run

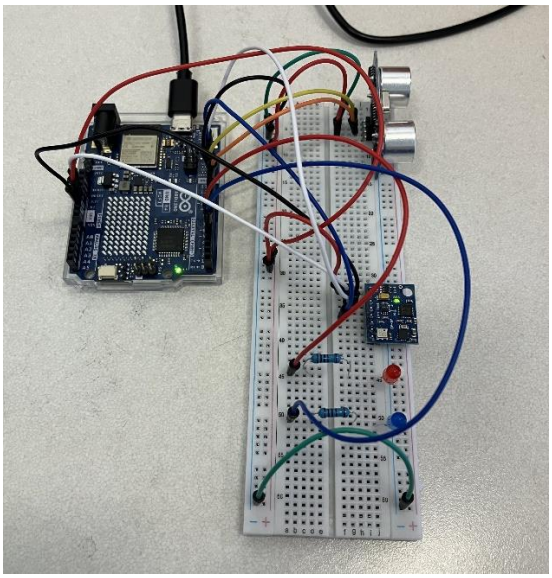
and use its collected data to calculate the distances. These distances were then stored in the JSON object. Each of the functions to gather the data from the GY-87 sensors were also run in this loop. The JSON object was then serialized and sent to Node-Red.

Creating the graphical user interfaces in Node-Red was the next step. A flow was created that received a JSON object from Arduino using a serial in node. The JSON object was then converted to a JavaScript object that Node-Red could understand. Multiple function nodes were then connected. Each function node collected a specific piece of data from the object. Each piece of data was then visualized in its own way. Distance from the ultrasonic sensor was displayed in a chart. Temperature, pressure, and altitude were displayed in a table. Acceleration and rotation were displayed as gauges. Azimuth and direction were displayed as compasses, which is a type of gauge. A second flow was also created in Node-Red. Two inject nodes were placed, one for the red LED and one for the blue LED. These inputs were then joined together and converted to a JSON formatted string. The JSON string was then sent to the IDE using a serial out node. Back in the IDE, in the loop, if there was data sent from Node-Red, the incoming JSON object was read and converted into data the IDE could understand. If a 1 was sent, the red LED was turned on and the blue LED's intensity was set to the value sent from Node-Red. If a 0 was sent, the red LED was turned off and the blue LED's intensity was set to the value sent from Node-Red.

Results

The project was successful. The sensors were all able to collect the necessary data. The data looked reasonable based on the environment. The Arduino was able to read, process, and send the data to Node-Red for further processing. Node-Red was able to read, process, and display the data. It looked like the GUIs visualized the data correctly. Sending data from Node-Red to the Arduino was successful as well. The Arduino was able to receive and interpret the JSON string and the LEDs reacted correctly based on the inputs sent from Node-Red. The red LED was turned on or off depending on if a 1 or 0 was sent and the blue LED's intensity was changed depending on the value between 0 and 255 that was sent.

Picture of hardware:



Appendix

Arduino code:

```
// Include required libraries
#include <Adafruit_MPU6050.h>
#include <Adafruit_Sensor.h>
#include <Wire.h>
#include <Adafruit_BMP085.h>
#include <QMC5883LCompass.h>
#include <ArduinoJson.h> // Import ArduinoJson library

#define TRIG_PIN  12 // Arduino pin connected to sensor TRIGGER pin
#define ECHO_PIN  13 // Arduino pin connected to sensor ECHO pin

float duration_us, distance_cm;
JsonDocument doc;

// Initialize sensor objects
Adafruit_MPU6050 mpu;
Adafruit_BMP085 bmp;
QMC5883LCompass compass;

// initialize pins for each LED
int red = 6;
int blue = 5;

void setup() {
  // Initialize the serial communication with a baud rate of 9600
  Serial.begin(9600);

  pinMode(TRIG_PIN, OUTPUT); // configure the trigger pin to output mode
  pinMode(ECHO_PIN, INPUT); // configure the echo pin to input mode
  pinMode(red, OUTPUT);
  pinMode(blue, OUTPUT);

  // Initialize the MPU6050 sensor (accelerometer and gyroscope)
  initializeMPU6050();

  // Enable I2C bypass on MPU6050 to directly access the QMC5883L magnetometer
  mpu.setI2CBypass(true);

  // Initialize the QMC5883L magnetometer sensor
  initializeQMC5883L();

  // Initialize BMP180 barometric sensor
```

```
  initializeBMP180();
}

void loop() {
  delay (1000);
  // generate 10-microsecond pulse to TRIG pin
  digitalWrite(TRIG_PIN, HIGH);
  delayMicroseconds(10);
  digitalWrite(TRIG_PIN, LOW);

  duration_us = pulseIn(ECHO_PIN, HIGH); // Duration of pulse from ECHO pin
  distance_cm = 0.017 * duration_us; // calculate the distance

  doc["distance"] = distance_cm;

  // Print MPU6050 data
  printMPU6050();

  // Print QMC5883L data
  printQMC5883L();

  // Print BMP180 data
  printBMP180();

  serializeJson(doc, Serial);
  Serial.println();

  delay(500);

  if (Serial.available() > 0) {
    // parse incoming json string from node-red into json doc
    StaticJsonDocument<200> combination;
    String incoming = Serial.readString();
    deserializeJson(combination, incoming);

    if (combination["red"] == 1) {
      digitalWrite(red, HIGH);  // turn the LED on (HIGH is the voltage level)
      analogWrite(blue, combination["blue"]); // change the intensity of the LED
    } else {
      digitalWrite(red, LOW); // turn the LED off (LOW is the voltage level)
      analogWrite(blue, combination["blue"]); // change the intensity of the LED
    }
  }
}

// initialize and calibrate compass
```

```
void initializeQMC5883L() {

  compass.init();

  // calibrations from compass_calibration.ino
  compass.setCalibrationOffsets(-419.00, 1789.00, 4861.00);
  compass.setCalibrationScales(1.18, 1.73, 0.64);

}

// measure and add azimuth and direction values to json doc
void printQMC5883L() {

  int x, y, z, a;
  char myArray[3];

  compass.read();

  a = compass.getAzimuth();

  compass.getDirection(myArray, a);

  doc["azimuth"] = a;

  doc["direction"] = myArray;
}

void initializeMPU6050() {
  // Check if the MPU6050 sensor is detected
  if (!mpu.begin()) {
    Serial.println("Failed to find MPU6050 chip");
    while (1)
      ;  // Halt if sensor not found
  }

  // set accelerometer range to +-8G
  mpu.setAccelerometerRange(MPU6050_RANGE_8_G);

  // set gyro range to +- 500 deg/s
  mpu.setGyroRange(MPU6050_RANGE_500_DEG);

  // set filter bandwidth to 21 Hz
  mpu.setFilterBandwidth(MPU6050_BAND_21_HZ);

  delay(100);
}
```

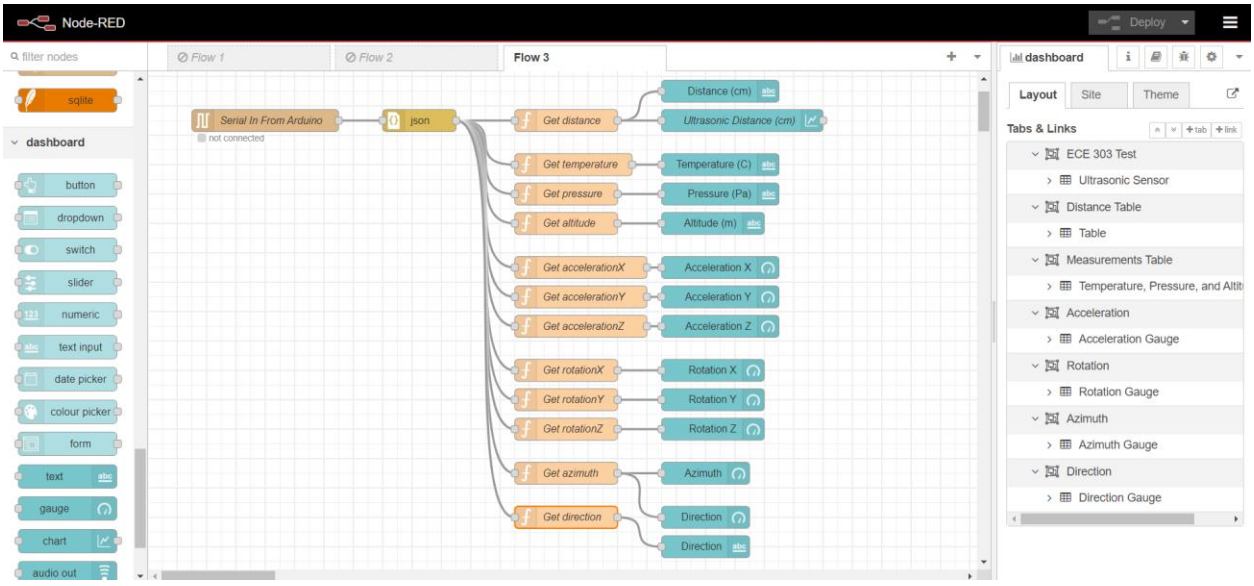
```
// measure and add acceleration and rotation values to json doc
void printMPU6050() {

  sensors_event_t a, g, temp;
  mpu.getEvent(&a, &g, &temp);

  doc["accelerationX"] = a.acceleration.x;
  doc["accelerationY"] = a.acceleration.y;
  doc["accelerationZ"] = a.acceleration.z;

  doc["rotationX"] = g.gyro.x;
  doc["rotationY"] = g.gyro.y;
  doc["rotationZ"] = g.gyro.z;
}

void initializeBMP180() {
  // Start BMP180 initialization
  if (!bmp.begin()) {
    Serial.println("Could not find a valid BMP180 sensor, check wiring!");
    while (1)
      ;  // Halt if sensor not found
  }
}

// measure and add temp, pressure, altitude, and sea level pressure values to json doc
void printBMP180() {

  doc["temperature"] = bmp.readTemperature();

  doc["pressure"] = bmp.readPressure();

  // Calculate altitude assuming 'standard' barometric
  // pressure of 1013.25 millibar = 101325 Pascal
  doc["altitude"] = bmp.readAltitude();

  doc["sealevelpressure"] = bmp.readSealevelPressure();
}
```

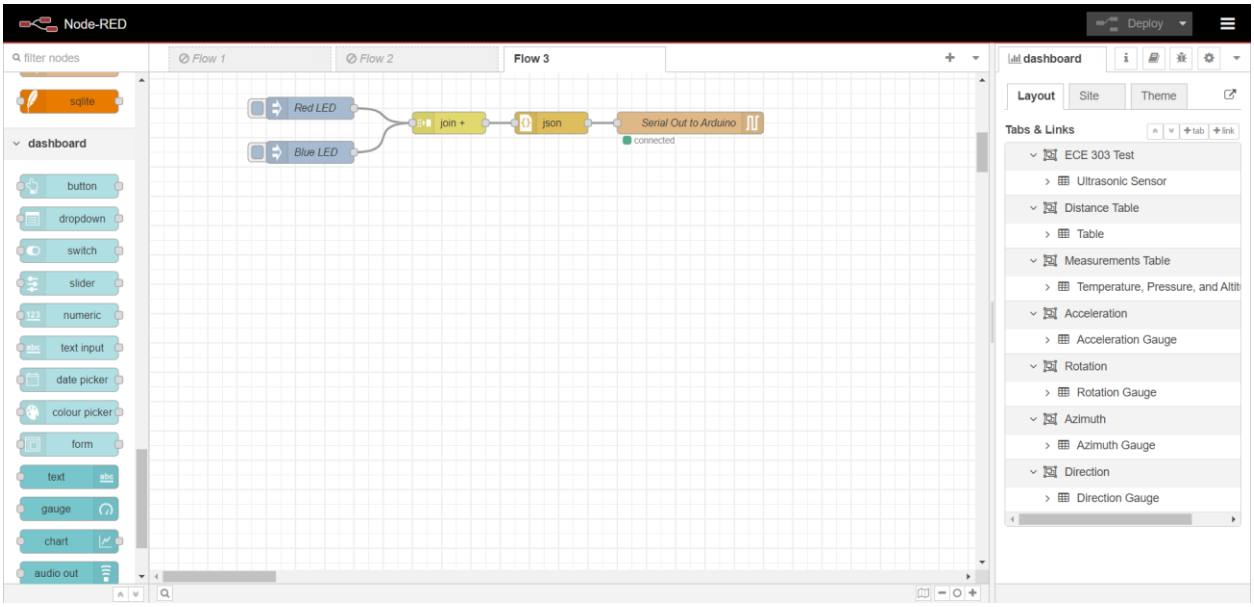
Serial Monitor output:

{"distance":55.097,"accelerationX":0.366313,"accelerationY":-0.155623,"accelerationZ":9.148245,"rotationX":-0.016787,"rotationY":-0.003997,"rotationZ":-0.029311,"azimuth":-77,"direction":"W","temperature":23.9,"pressure":101178,"altitude":11.32927,"sealevelpressure":101186}

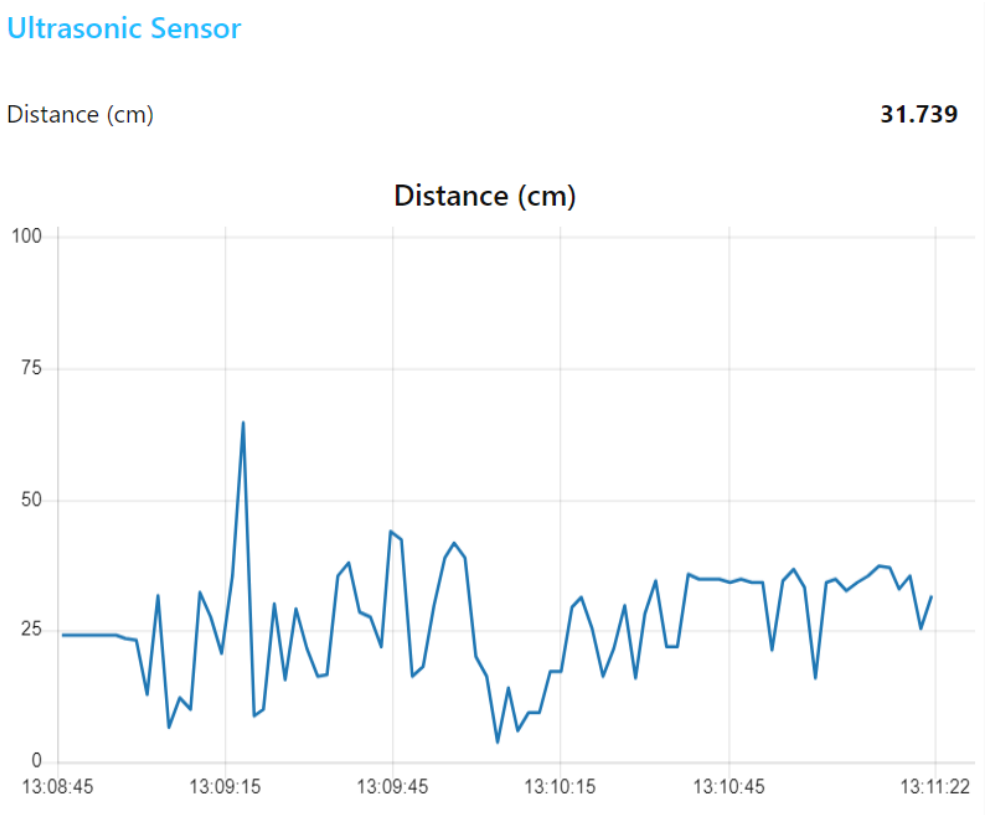
{"distance":55.437,"accelerationX":0.342371,"accelerationY":-0.1652,"accelerationZ":9.131485,"rotationX":-0.016254,"rotationY":-0.00373,"rotationZ":-0.028245,"azimuth":-77,"direction":"W","temperature":23.9,"pressure":101180,"altitude":12.24595,"sealevelpressure":101176}
{"distance":20.842,"accelerationX":0.550666,"accelerationY":-0.172383,"accelerationZ":9.141062,"rotationX":-0.015188,"rotationY":-0.004796,"rotationZ":0.376245,"azimuth":-42,"direction":"NW","temperature":23.9,"pressure":101177,"altitude":11.99575,"sealevelpressure":101181}
{"distance":50.694,"accelerationX":0.320823,"accelerationY":-0.150835,"accelerationZ":9.148245,"rotationX":-0.018119,"rotationY":-0.001599,"rotationZ":-0.090597,"azimuth":-83,"direction":"W","temperature":23.9,"pressure":101184,"altitude":12.07932,"sealevelpressure":101179}
{"distance":48.688,"accelerationX":0.289698,"accelerationY":0.083797,"accelerationZ":9.11712,"rotationX":-0.01359,"rotationY":-0.01279,"rotationZ":0.567832,"azimuth":-90,"direction":"W","temperature":23.9,"pressure":101177,"altitude":12.07932,"sealevelpressure":101182}
{"distance":45.356,"accelerationX":0.560243,"accelerationY":-0.11971,"accelerationZ":9.150639,"rotationX":-0.018919,"rotationY":0.007994,"rotationZ":-1.38374,"azimuth":-91,"direction":"WSW","temperature":23.9,"pressure":101178,"altitude":12.07932,"sealevelpressure":101189}
{"distance":38.998,"accelerationX":0.337582,"accelerationY":-0.134075,"accelerationZ":9.13388,"rotationX":-0.017054,"rotationY":-0.003464,"rotationZ":-0.03011,"azimuth":-74,"direction":"W","temperature":23.9,"pressure":101177,"altitude":11.41232,"sealevelpressure":101180}
{"distance":38.998,"accelerationX":0.339977,"accelerationY":-0.155623,"accelerationZ":9.153033,"rotationX":-0.015721,"rotationY":-0.00373,"rotationZ":-0.028511,"azimuth":-74,"direction":"W","temperature":23.9,"pressure":101184,"altitude":11.66251,"sealevelpressure":101190}

Node-Red flows:





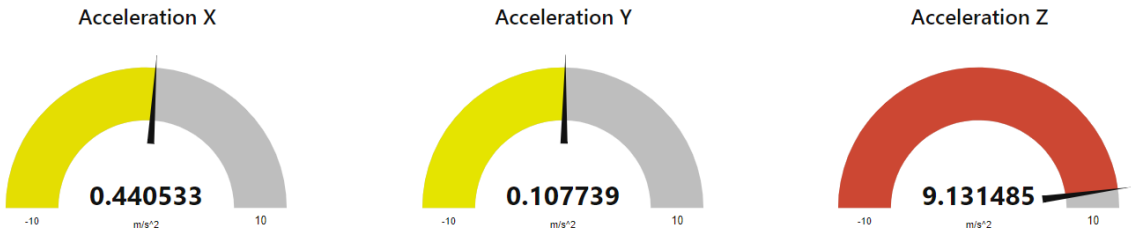
Node-Red GUIs:



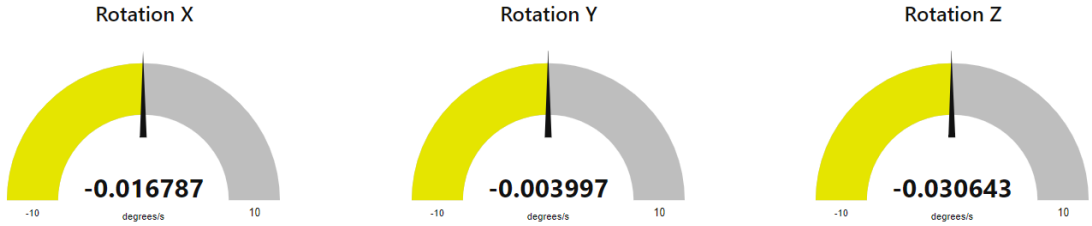
Temperature, Pressure, and Altitude Table

Temperature (C)	**24.6**	Pressure (Pa)	**101364**	Altitude (m)	**-3.662937**

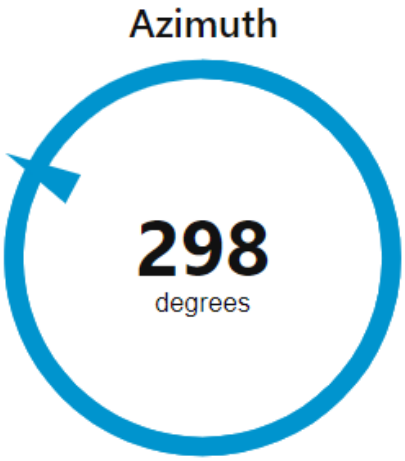
Acceleration Gauge



Rotation Gauge



Azimuth Gauge



Direction Gauge

Direction **NE**

Subject: Assignment 4 – Introduction to Wireless

To: Christopher Peters, PhD.
Teaching Professor, Electrical and Computer Engineering Department
Drexel University

From: Sofia D'Alessandro
Undergraduate, College of Engineering
Drexel University

Purpose

The purpose of this memo is to demonstrate the concept of wireless communication. This concept was demonstrated in this project by having two Arduino Uno R4 WiFis communicate with each other over a wireless connection. One Arduino acted as a server and the other Arduino acted as a client. HTTP requests were sent from the client to the server in the form of pressing either a yellow or blue button on the client side to turn on or turn off a yellow or blue LED, respectively, on the server side. The server accepted the request and responded in the form of turning on or turning off the corresponding LED.

Methodology/Approach

In conducting the lab, the first step was to physically build the hardware using two solderless breadboards, one yellow LED, one blue LED, one yellow button, one blue button, jumper wires, four 1-kohm resistors, and two Arduino Uno R4 WiFis. Two different circuits were built using a different breadboard and Arduino. The Arduino in the circuit with the LEDs acted as the server, while the Arduino in the circuit with the buttons acted as the client. Each LED was connected to a specific digital pin on the server Arduino and each button was connected to a specific digital pin on the client Arduino.

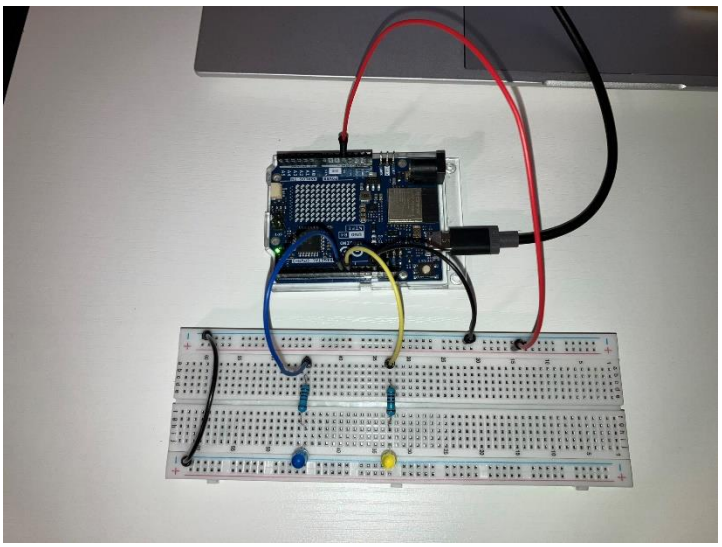
After the circuit was built, the next step was to begin working in the Arduino IDE. Two different programs were written, one for the server and one for the client. The server program was written first. First, the required WiFi library and necessary variables, such as the name of the network SSID and its password, were initialized. A web server was created at port 80. Communication with the serial port was initialized. Next, the setup of the wireless connection began. An access point, which allows wireless devices to connect to a wireless server, was created and given an IP address. The access point and network were then started. The server was then allowed to host devices. Inside the loop, the server checked if a client connected and if it sent an HTTP request. The loop checked for four different types of requests, turning the yellow LED on, turning the yellow LED off, turning the blue LED on, and turning the blue LED off, and then performed the action based on which one it received. When the request was completed, the client was then disconnected. When the server program was finished, the client program began. This program began the same way as the server program. The required WiFi library and necessary variables were initialized. The setup was also similar. The client was initialized and connection to the serial port was initialized. Then next step was to connect to the server using the SSID and its password. In the loop, four different checks occurred. Each one checked if a button was pushed

and what its corresponding LED's current state was at the time of it being pushed. If the button was pushed, the LED's state was toggled. The client then connected to the server to send its HTTP request so that the action could be carried out.

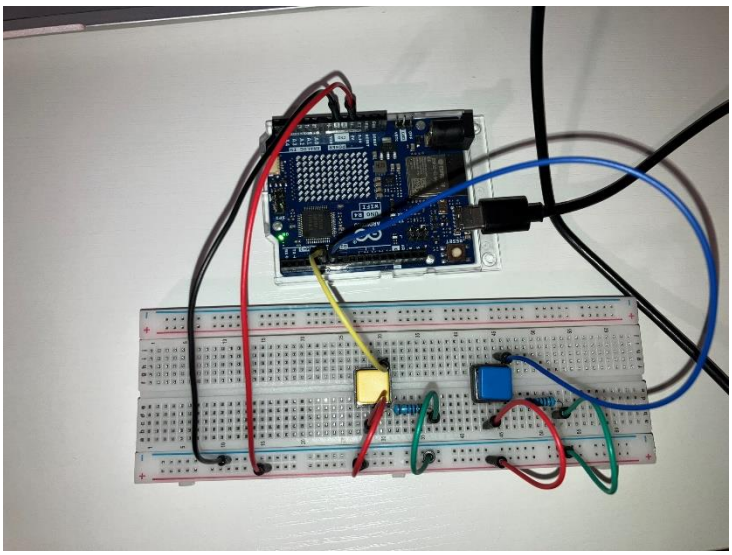
Results

The project was successful. The server Arduino was able to create and start the access point and server. The client Arduino was able to connect to the server. The client Arduino was able to read that either the yellow or blue button was pushed and then send an HTTP request to the server to either turn on or turn off the yellow or blue LED. The server Arduino was able to read the request and grant it by turning on or turning off the necessary LED depending on which button was pushed and what the LED's state was at the time of it being pushed.

Picture of hardware:



Server



Client

Serial Monitor output:

(Server)

```
Please upgrade the firmware
Creating access point named: SDserver
Waiting for connection
SSID: SDserver
IP Address: 192.48.56.2
Device connected to AP
new client
GET /led1on HTTP/1.1

client disconnected
new client
GET /led2on HTTP/1.1

client disconnected
new client
GET /led1off HTTP/1.1

client disconnected
new client
GET /led2off HTTP/1.1

client disconnected
```

(Client)

```
Connected to server
```

Appendix

Arduino code:

(Server)

```
#include "WiFiS3.h"

char ssid[] = "SDserver";        // your network SSID (name)
char pass[] = "password!"; // your network password (use for WPA, or use as key for WEP)
int keyIndex = 0;                // your network key index number (needed only for WEP)

int YELLOW_LED =  10;  // Pin number for Yellow LED
int BLUE_LED = 9; // Pin number for Blue LED
bool YELLOW_LED_STATUS = LOW; // Initial Status of Yellow LED pin
bool BLUE_LED_STATUS = LOW; // Initial Status of Blue LED pin
```

```
int status = WL_IDLE_STATUS;  // Initial Status of WiFi connection

WiFiServer server(80); // Create the web server on port 80

void setup() {
  //Initialize serial and wait for port to open:
  Serial.begin(9600);

  while (!Serial) {
    ; // wait for serial port to connect. Needed for native USB port only
  }
  Serial.println("Access Point Web Server");

  pinMode(YELLOW_LED, OUTPUT); // set the Yellow LED pin mode
  pinMode(BLUE_LED, OUTPUT); // set the Blue LED pin mode

  // check for the WiFi module:
  if (WiFi.status() == WL_NO_MODULE) {
    Serial.println("Communication with WiFi module failed!");
    // don't continue
    while (true);
  }

  // check firmware version
  String fv = WiFi.firmwareVersion();
  if (fv < WIFI_FIRMWARE_LATEST_VERSION) {
    Serial.println("Please upgrade the firmware");
  }

  // by default the local IP address will be 192.168.4.1
  // you can override it with the following:
  WiFi.config(IPAddress(192,48,56,2));

  // print the network name (SSID);
  Serial.print("Creating access point named: ");
  Serial.println(ssid);

  // Create open network
  status = WiFi.beginAP(ssid, pass);
  if (status != WL_AP_LISTENING) {
    Serial.println("Creating access point failed");
    // don't continue
    while (true);
  }
  Serial.println("Waiting for connection");
  // wait 10 seconds for connection:
```

```
  delay(10000);

  // start the web server on port 80
  server.begin();

  // you're connected now, so print out the status
  printWiFiStatus();

}

void loop() {

  // compare the previous status to the current status
  if (status != WiFi.status()) {
    // it has changed update the variable
    status = WiFi.status();

    if (status == WL_AP_CONNECTED) {
      // a device has connected to the AP
      Serial.println("Device connected to AP");
    } else {
      // a device has disconnected from the AP, and we are back in listening mode
      Serial.println("Device disconnected from AP");
    }
  }

  WiFiClient client = server.available();   // listen for incoming clients

  if (client) {                             // if you get a client,
    Serial.println("new client");           // print a message out the serial port
    String currentLine = "";                // make a String to hold incoming data from the client
    while (client.connected()) {            // loop while the client's connected
      delayMicroseconds(10);                // This is required for the Arduino Nano RP2040 Connect -
otherwise it will loop so fast that SPI will never be served.
      if (client.available()) {             // if there's bytes to read from the client,
        char c = client.read();             // read a byte, then
        Serial.write(c);                    // print it out to the serial monitor
        if (c == '\n') {                    // if the byte is a newline character

          // if the current line is blank, you got two newline characters in a row.
          // that's the end of the client HTTP request, so send a response:
          if (currentLine.length() == 0) {
            // HTTP headers always start with a response code (e.g. HTTP/1.1 200 OK)
            // and a content-type so the client knows what's coming, then a blank line:
            client.println("HTTP/1.1 200 OK");
            client.println("Content-type:text/html");
```

```
          client.println();

          // The HTTP response ends with another blank line:
          client.println();
          // break out of the while loop:
          break;
        }
        else {
          // Check to see if the client request was "GET /led1on":
          if (currentLine.startsWith("GET /led1on")) {
            digitalWrite(YELLOW_LED, HIGH);  // GET /led1on turns the Yellow LED on
            YELLOW_LED_STATUS = HIGH; // toggle yellow LED status

            // Check to see if the client request was "GET /led1off":
          } else if (currentLine.startsWith("GET /led1off")) { // Check to see if the client request
was "GET /led1off":
            digitalWrite(YELLOW_LED, LOW);  // GET /led1off turns the Yellow LED off
            YELLOW_LED_STATUS = LOW;  // toggle yellow LED status

            // Check to see if the client request was "GET /led2on":
          } else if (currentLine.startsWith("GET /led2on")) {  // Check to see if the client request
was "GET /led2on":
            digitalWrite(BLUE_LED, HIGH); // GET /led2on turns the Blue LED on
            BLUE_LED_STATUS = HIGH; // toggle blue LED status

            // Check to see if the client request was "GET /led2off":
          } else if (currentLine.startsWith("GET /led2off")) {
            digitalWrite(BLUE_LED, LOW);  // GET /led2off turns the Blue LED off
            BLUE_LED_STATUS = LOW;  // toggle blue LED status
          }
          // if you got a newline, then clear currentLine:
          currentLine = "";
        }
      }
      else if (c != '\r') {    // if you got anything else but a carriage return character,
        currentLine += c;      // add it to the end of the currentLine
      }
    }
  }
  // close the connection:
  client.stop();
  Serial.println("client disconnected");
 }
}

void printWiFiStatus() {
```

```
  // print the SSID of the network you're attached to:
  Serial.print("SSID: ");
  Serial.println(WiFi.SSID());

  // print your WiFi shield's IP address:
  IPAddress ip = WiFi.localIP();
  Serial.print("IP Address: ");
  Serial.println(ip);
}
```

(Client)

```
#include "WiFiS3.h"

char ssid[] = "SDserver";        // your network SSID (name)
char pass[] = "password!"; // your network password (use for WPA, or use as key for WEP)
int keyIndex = 0;                // your network key index number (needed only for WEP)

int YELLOW_BUTTON =  5;  // Pin number for Yellow Button
int BLUE_BUTTON = 6; // Pin number for Blue Button
bool YELLOW_LED_STATUS = LOW; // Initial Status of Yellow LED pin
bool BLUE_LED_STATUS = LOW; // Initial Status of Blue LED pin
int status = WL_IDLE_STATUS;  // Initial Status of WiFi connection
bool YELLOW_BUTTON_STATUS = LOW; // Initial Status of Yellow Button pin
bool BLUE_BUTTON_STATUS = LOW; // Initial Status of Blue Button pin

WiFiClient client;

void setup() {
  //Initialize serial and wait for port to open:
  Serial.begin(9600);

  while (!Serial) {
    ; // wait for serial port to connect. Needed for native USB port only
  }

  pinMode(YELLOW_BUTTON, INPUT); // set the Yellow button pin mode
  pinMode(BLUE_BUTTON, INPUT); // set the Blue button pin mode

  // Connect to server
  status = WiFi.begin(ssid, pass);
  if (status != WL_CONNECTED) {
    Serial.println("Connection to server failed");
    // don't continue
    while (true);
  }
```

```
  Serial.println("Waiting for connection");
  // wait 10 seconds for connection:
  delay(10000);

  Serial.println("Connected to server");
  delay(5000);
}

void loop() {

  // Check to see if the yellow button was pushed and what the LED's state was
  if (digitalRead(YELLOW_BUTTON) == HIGH && YELLOW_LED_STATUS == LOW) {
    // send HTTP request to turn yellow LED on
    if (client.connect(IPAddress(192, 48, 56, 2), 80)) {
      client.println("GET /led1on HTTP/1.1");
      client.println();
    }
    YELLOW_LED_STATUS = HIGH;  // toggle state of yellow LED
    delay(500);
  }

  // Check to see if the yellow button was pushed and what the LED's state was
  if (digitalRead(YELLOW_BUTTON) == HIGH && YELLOW_LED_STATUS == HIGH) {
    // send HTTP request to turn yellow LED off
    if (client.connect(IPAddress(192, 48, 56, 2), 80)) {
      client.println("GET /led1off HTTP/1.1");
      client.println();
    }
    YELLOW_LED_STATUS = LOW; // toggle state of yellow LED
    delay(500);
  }

  // Check to see if the button button was pushed and what the LED's state was
  if (digitalRead(BLUE_BUTTON) == HIGH && BLUE_LED_STATUS == LOW) {
    // send HTTP request to turn blue LED on
    if (client.connect(IPAddress(192, 48, 56, 2), 80)) {
      client.println("GET /led2on HTTP/1.1");
      client.println();
    }
    BLUE_LED_STATUS = HIGH;  // toggle state of blue LED
    delay(500);
  }

  // Check to see if the button button was pushed and what the LED's state was
  if (digitalRead(BLUE_BUTTON) == HIGH && BLUE_LED_STATUS == HIGH) {
    // send HTTP request to turn blue LED off
```

```
    if (client.connect(IPAddress(192, 48, 56, 2), 80)) {
      client.println("GET /led2off HTTP/1.1");
      client.println();
    }
    BLUE_LED_STATUS = LOW; // toggle state of blue LED
    delay(500);
  }
}
```

Subject: Assignment 5 – GET and POST Implementation

To: Christopher Peters, PhD.
Teaching Professor, Electrical and Computer Engineering Department
Drexel University

From: Sofia D'Alessandro
Undergraduate, College of Engineering
Drexel University

Purpose

The purpose of this memo is to demonstrate implementing GET and POST requests using wireless communication to affect the behavior of LEDs. This project built off of Assignment 3 and Assignment 4. It involved using HTTP requests to achieve the same functionality from Assignment 3. The project was divided into three parts. The first part required the Arduino to act as a wireless server and receive a GET request. The second part required the Arduino to act as a wireless client and send a POST request. The third part required Python to send a GET request.

Methodology/Approach

In conducting the lab, the first step was to physically build the circuit that would be used in all three parts. It was the same circuit used in Assignment 3. The circuit was built using a solderless breadboard, the Arduino Uno R4 WiFi, the HC-SR04 ultrasonic sensor, the GY-87 sensor, one red LED, one blue LED, two 1000-ohm resistors, and jumper wires. The ultrasonic sensor was placed on the breadboard. The power rail of the breadboard was connected to the 5V pin on the Arduino and then the ultrasonic sensor's Vcc pin and the GY-87 sensor's Vcc_in pin were connected to the power rail. The ground rail was connected to a ground pin on the Arduino and then the ultrasonic sensor's ground pin was connected to the ground rail. The ultrasonic sensor's trig pin was connected to pin 12 on the Arduino and the ultrasonic sensor's echo pin was connected to pin 13 on the Arduino. The GY-87 sensor's ground pin was connected to a ground pin on the Arduino and the GY-87 sensor's 3.3V pin was connected to the 3.3V pin on the Arduino. The GY-87 sensor's SCL pin was connected to the SCL pin on the Arduino and the GY-87 sensor's SDA pin was connected to the SDA pin on the Arduino. The red and blue LEDS were then placed on the breadboard. Their cathodes were connected to the other ground rail while their anodes were connected to the resistors, which were then connected to pin 6 and pin 5, respectively. Finally, both ground rails were connected.

After the circuit was built, work in the IDE and Node-Red could begin. Two different programs and flows were written and created: one for when the Arduino acted as the server receiving the GET request from Node-RED and the other for when the Arduino acted as the client sending the POST request to Node-RED. The program for Part A was created first. The program began similarly to how Assignment 4 began, with setting up the wireless connection. The access pointed was created and started up. The red and blue LEDs were initialized and the functions to initialize and print the data from the primary sensors in the GY-87 sensor that were used in Assignment 3 were implemented. Inside the loop, the server checked if a client connected. If a client connected, it checked if a GET request was sent. It check the parameters of the GET request. It checked if the request called for the blue LED to be turned on or off and it checked

what intensity was wanted for the red LED. The LEDs were then adjusted based on what was specified in the request. When the request was finished, a HTTP response was sent back to the client. Along with the response, the ultrasonic sensor took a reading and stored it in the JSON object. Then the functions for the GY-87 sensors were called and those measurements were recorded and stored in the JSON object as well. The JSON object was then sent to the Node-RED. In Node-RED, a flow was created to send the GET request to the Arduino and also to display the results from the response from the Arduino. The Node-RED flow from Assignment 3 was modified to accomplish this. Two input nodes were added: one for the blue LED and the other for the red LED. These nodes were connected to their respective function nodes to extract the inputs for each LED. The inputs were then joined together into one object. The join node was then connected to a function node that used the inputs to construct the GET request. The url for the GET request was created and then sent to the HTTP request node, which would be sent to the Arduino. The HTTP request node would then return a parsed JSON object, which was the sensor data sent from the Arduino. The data was then converted to a JavaScript object that Node-RED could understand and process. Each piece of data was then visualized in its own way. An inject node was then connected to the function node that constructed the GET request so a request could repeatedly be created and sent every ten seconds.

The program for Part B was written next. The beginning of this program was the same as the program for Part A. The access pointed was created and started up. The red and blue LEDs were initialized and the functions to initialize and print the data from the primary sensors in the GY-87 sensor were implemented. Inside the loop, the server checked if a client connected. If a client connected, it checked if a GET request was sent. It check the parameters of the GET request. It checked if the request called for the blue LED to be turned on or off and it checked what intensity was wanted for the red LED. The LEDs were then adjusted based on what was specified in the request. A function was written to gather all of the sensor data from the ultrasonic sensor and the GY-87 sensor and store it in the JSON object. Another function was written to send all of the data in the JSON object to Node-RED using a POST request. This POST request was then sent every ten seconds. The Node-RED flow was then created. The flow from Part A was copied for Part B to send a GET request to the Arduino for the LED input values as a subflow. Another subflow was responsible for receiving the POST request. The data sent from the Arduino in the POST request was then converted to a JavaScript object that Node-RED could understand and process. Each piece of data was then visualized in its own way.

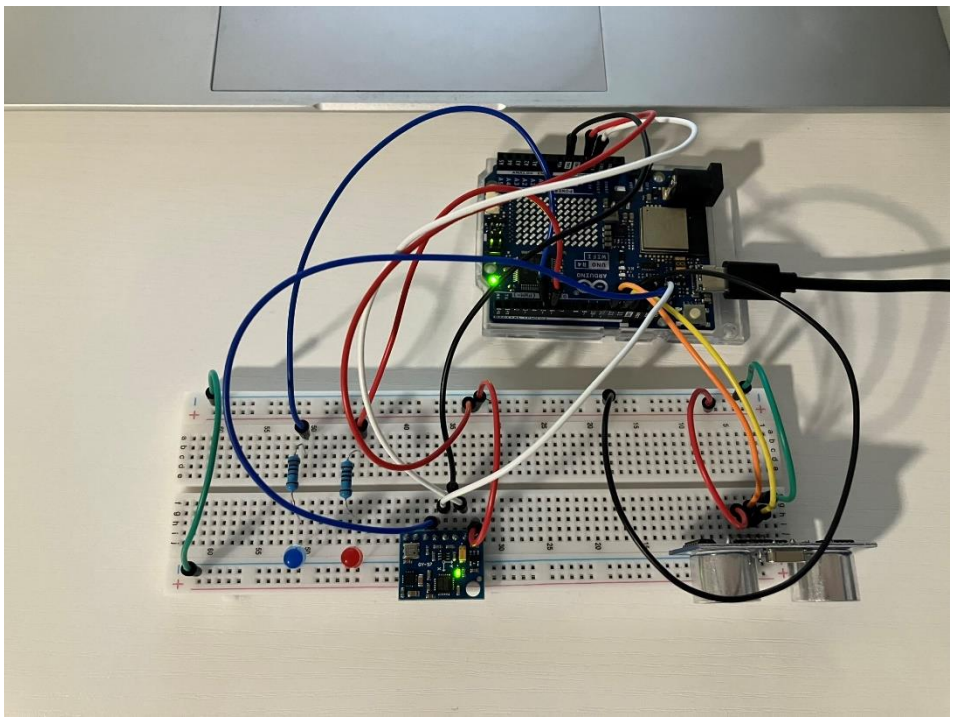
With the Arduino programs completed, the Python program for Part C was then written. This Python program worked with the Arduino code from Part A to send a GET request. Two inputs were used to receive the parameters for the blue and red LEDs. These parameters were then imbedded into the URL which would be used for the GET request. Another input was then used to send the request to the URL. A function was also written to get information about the response, such as the URL, Content, Status, etc.

Results

The project was successful. The Arduino was able to receive a GET request from Node-RED with inputs for the blue and red LEDs. The Arduino was able to turn the blue LED on and off and change the intensity of the red LED based on the parameters given in the request. The Arduino was also able to act as a wireless client and send a POST request to Node-RED. The data from the sensors was able to be sent from the Arduino to Node-RED and was displayed properly in

each respective GUI. The Python part of the project was also successful. Python was able to send a GET request containing the parameters for the LEDs to the Arduino and the LEDs were able to respond correctly.

Picture of hardware:



Output of Python output:

Enter the blue value (0-1): 1
Enter the red value (0-255): 255
Press ENTER to Send Request to http://192.48.56.2/blue=1&red=255
URL: http://192.48.56.2/blue=1&red=255
Apparent Encoding: ascii
Encoding: ascii
Content:
b'\r\n{"distance":30.719,"accelerationX":0.387861,"accelerationY":0.009577,"accelerationZ":9.229648,"rotationX":-0.017853,"rotationY":-0.005329,"rotationZ":-0.028778,"azimuth":-87,"direction":"
W","temperature":27.7,"pressure":101280,"altitude":3.580727,"sealevelpressure":101282}'
Headers: {'Content-type': 'application/json'}
Status Code: 200
Data: {'distance': 30.719, 'accelerationX': 0.387861, 'accelerationY': 0.009577, 'accelerationZ': 9.229648, 'rotationX': -0.017853, 'rotationY': -0.005329, 'rotationZ': -0.028778, 'azimuth': -87, 'direction': ' W', 'temperature': 27.7, 'pressure': 101280, 'altitude': 3.580727, 'sealevelpressure': 101282}

Appendix

Arduino code:

(Part A and Part C)

```
// Include required libraries
#include <Adafruit_MPU6050.h>
#include <Adafruit_Sensor.h>
#include <Wire.h>
#include <Adafruit_BMP085.h>
#include <QMC5883LCompass.h>
#include <ArduinoJson.h> // Import ArduinoJson library
#include "WiFiS3.h"

char ssid[] = "SDserver";        // your network SSID (name)
char pass[] = "password!"; // your network password (use for WPA, or use as key for WEP)
int keyIndex = 0;                // your network key index number (needed only for WEP)

int status = WL_IDLE_STATUS;  // Initial Status of WiFi connection

WiFiServer server(80); // Create the web server on port 80

#define TRIG_PIN  12 // Arduino pin connected to sensor TRIGGER pin
#define ECHO_PIN  13 // Arduino pin connected to sensor ECHO pin

float duration_us, distance_cm;
JsonDocument doc;

// Initialize sensor objects
Adafruit_MPU6050 mpu;
Adafruit_BMP085 bmp;
QMC5883LCompass compass;

// initialize pins for each LED
int red = 6;
int blue = 5;

void setup() {
  // Initialize the serial communication with a baud rate of 9600
  Serial.begin(9600);

  while (!Serial) {
    ; // wait for serial port to connect. Needed for native USB port only
  }
  Serial.println("Access Point Web Server");

  pinMode(TRIG_PIN, OUTPUT); // configure the trigger pin to output mode
  pinMode(ECHO_PIN, INPUT); // configure the echo pin to input mode
```

```
pinMode(red, OUTPUT);
pinMode(blue, OUTPUT);

// check for the WiFi module:
if (WiFi.status() == WL_NO_MODULE) {
  Serial.println("Communication with WiFi module failed!");
  // don't continue
  while (true);
}

// check firmware version
String fv = WiFi.firmwareVersion();
if (fv < WIFI_FIRMWARE_LATEST_VERSION) {
  Serial.println("Please upgrade the firmware");
}

// by default the local IP address will be 192.168.4.1
// you can override it with the following:
WiFi.config(IPAddress(192,48,56,2));

// print the network name (SSID);
Serial.print("Creating access point named: ");
Serial.println(ssid);

// Create open network
status = WiFi.beginAP(ssid, pass);
if (status != WL_AP_LISTENING) {
  Serial.println("Creating access point failed");
  // don't continue
  while (true);
}
Serial.println("Waiting for connection");
// wait 10 seconds for connection:
delay(10000);

// start the web server on port 80
server.begin();

// you're connected now, so print out the status
printWiFiStatus();

// Initialize the MPU6050 sensor (accelerometer and gyroscope)
initializeMPU6050();

// Enable I2C bypass on MPU6050 to directly access the QMC5883L magnetometer
mpu.setI2CBypass(true);
```

```
  // Initialize the QMC5883L magnetometer sensor
  initializeQMC5883L();

  // Initialize BMP180 barometric sensor
  initializeBMP180();
}

void loop() {

  // compare the previous status to the current status
  if (status != WiFi.status()) {
    // it has changed update the variable
    status = WiFi.status();

    if (status == WL_AP_CONNECTED) {
      // a device has connected to the AP
      Serial.println("Device connected to AP");
    } else {
      // a device has disconnected from the AP, and we are back in listening mode
      Serial.println("Device disconnected from AP");
    }
  }

  WiFiClient client = server.available();   // listen for incoming clients

  if (client) {                             // if you get a client,
    doc.clear();
    Serial.println("new client");           // print a message out the serial port
    String currentLine = "";                // make a String to hold incoming data from the client
    while (client.connected()) {            // loop while the client's connected
      delayMicroseconds(10);                // This is required for the Arduino Nano RP2040 Connect -
otherwise it will loop so fast that SPI will never be served.
      if (client.available()) {             // if there's bytes to read from the client,
        char c = client.read();             // read a byte, then
        Serial.write(c);                    // print it out to the serial monitor
        if (c == '\n') {                    // if the byte is a newline character

          // if the current line is blank, you got two newline characters in a row.
          // that's the end of the client HTTP request, so send a response:
          if (currentLine.length() == 0) {
            // HTTP headers always start with a response code (e.g. HTTP/1.1 200 OK)
            // and a content-type so the client knows what's coming, then a blank line:
            client.println("HTTP/1.1 200 OK");
            client.println("Content-type: application/json");
            client.println();
```

```
          // The HTTP response ends with another blank line:
          client.println();

          delay (1000);
          // generate 10-microsecond pulse to TRIG pin
          digitalWrite(TRIG_PIN, HIGH);
          delayMicroseconds(10);
          digitalWrite(TRIG_PIN, LOW);

          duration_us = pulseIn(ECHO_PIN, HIGH); // Duration of pulse from ECHO pin
          distance_cm = 0.017 * duration_us; // calculate the distance

          doc["distance"] = distance_cm;

          // Print MPU6050 data
          printMPU6050();

          // Print QMC5883L data
          printQMC5883L();

          // Print BMP180 data
          printBMP180();

          serializeJson(doc, client);
          Serial.println();

          delay(500);

          // break out of the while loop:
          break;
        }
        else {

          if (currentLine.startsWith("GET /blue=1")) {
            digitalWrite(blue, HIGH);  // GET /blue=1 turns the blue LED on
            int index = currentLine.indexOf("red="); // find index of red=
            int intensity = currentLine.substring(index+4).toInt(); // find value of intensity for red
LED given in request
            analogWrite(red, intensity); // change the intensity of the red LED
          }

          if (currentLine.startsWith("GET /blue=0")) {
            digitalWrite(blue, LOW);  // GET /blue=0 turns the blue LED off
            int index = currentLine.indexOf("red="); // find index of red=
```

```
            int intensity = currentLine.substring(index+4).toInt(); // find value of intensity for red
LED given in request
            analogWrite(red, intensity); // change the intensity of the red LED
           }

           // if you got a newline, then clear currentLine:
           currentLine = "";
         }
       } else if (c != '\r') {    // if you got anything else but a carriage return character,
         currentLine += c;      // add it to the end of the currentLine
       }
     }
    }

    client.stop();
    Serial.println("client disconnected");
  }
}

// initialize and calibrate compass
void initializeQMC5883L() {

  compass.init();

  // calibrations from compass_calibration.ino
  compass.setCalibrationOffsets(-419.00, 1789.00, 4861.00);
  compass.setCalibrationScales(1.18, 1.73, 0.64);

}

// measure and add azimuth and direction values to json doc
void printQMC5883L() {

  int x, y, z, a;
  char myArray[3];

  compass.read();

  a = compass.getAzimuth();

  compass.getDirection(myArray, a);

  doc["azimuth"] = a;

  doc["direction"] = myArray;
}
```

```
void initializeMPU6050() {
  // Check if the MPU6050 sensor is detected
  if (!mpu.begin()) {
    Serial.println("Failed to find MPU6050 chip");
    while (1)
      ;  // Halt if sensor not found
  }

  // set accelerometer range to +-8G
  mpu.setAccelerometerRange(MPU6050_RANGE_8_G);

  // set gyro range to +- 500 deg/s
  mpu.setGyroRange(MPU6050_RANGE_500_DEG);

  // set filter bandwidth to 21 Hz
  mpu.setFilterBandwidth(MPU6050_BAND_21_HZ);

  delay(100);
}

// measure and add acceleration and rotation values to json doc
void printMPU6050() {

  sensors_event_t a, g, temp;
  mpu.getEvent(&a, &g, &temp);

  doc["accelerationX"] = a.acceleration.x;
  doc["accelerationY"] = a.acceleration.y;
  doc["accelerationZ"] = a.acceleration.z;

  doc["rotationX"] = g.gyro.x;
  doc["rotationY"] = g.gyro.y;
  doc["rotationZ"] = g.gyro.z;
}

void initializeBMP180() {
  // Start BMP180 initialization
  if (!bmp.begin()) {
    Serial.println("Could not find a valid BMP180 sensor, check wiring!");
    while (1)
      ;  // Halt if sensor not found
  }
}

// measure and add temp, pressure, altitude, and sea level pressure values to json doc
```

```
void printBMP180() {

  doc["temperature"] = bmp.readTemperature();

  doc["pressure"] = bmp.readPressure();

  // Calculate altitude assuming 'standard' barometric
  // pressure of 1013.25 millibar = 101325 Pascal
  doc["altitude"] = bmp.readAltitude();

  doc["sealevelpressure"] = bmp.readSealevelPressure();
}

void printWiFiStatus() {
  // print the SSID of the network you're attached to:
  Serial.print("SSID: ");
  Serial.println(WiFi.SSID());

  // print your WiFi shield's IP address:
  IPAddress ip = WiFi.localIP();
  Serial.print("IP Address: ");
  Serial.println(ip);
}
```

(Part B)

```
// Include required libraries
#include <Adafruit_MPU6050.h>
#include <Adafruit_Sensor.h>
#include <Wire.h>
#include <Adafruit_BMP085.h>
#include <QMC5883LCompass.h>
#include <ArduinoJson.h> // Import ArduinoJson library
#include "WiFiS3.h"

char ssid[] = "SDserver";        // your network SSID (name)
char pass[] = "password!"; // your network password (use for WPA, or use as key for WEP)
int keyIndex = 0;                // your network key index number (needed only for WEP)

int status = WL_IDLE_STATUS;  // Initial Status of WiFi connection

WiFiServer server(80); // Create the web server on port 80

#define TRIG_PIN  12 // Arduino pin connected to sensor TRIGGER pin
#define ECHO_PIN  13 // Arduino pin connected to sensor ECHO pin
```

```
float duration_us, distance_cm;
JsonDocument doc;

// Initialize sensor objects
Adafruit_MPU6050 mpu;
Adafruit_BMP085 bmp;
QMC5883LCompass compass;

// initialize pins for each LED
int red = 6;
int blue = 5;

// Node-RED IP address and port number
IPAddress nodeRedIP(192, 48, 56, 3);
const int nodeRedPort = 1880;

// Timer
unsigned long lastPost = 0;
const unsigned long postInterval = 10000;  // every 10 seconds

void setup() {
  // Initialize the serial communication with a baud rate of 9600
  Serial.begin(9600);

  while (!Serial) {
    ; // wait for serial port to connect. Needed for native USB port only
  }
  Serial.println("Access Point Web Server");

  pinMode(TRIG_PIN, OUTPUT); // configure the trigger pin to output mode
  pinMode(ECHO_PIN, INPUT); // configure the echo pin to input mode
  pinMode(red, OUTPUT);
  pinMode(blue, OUTPUT);

  // check for the WiFi module:
  if (WiFi.status() == WL_NO_MODULE) {
    Serial.println("Communication with WiFi module failed!");
    // don't continue
    while (true);
  }

  // check firmware version
  String fv = WiFi.firmwareVersion();
  if (fv < WIFI_FIRMWARE_LATEST_VERSION) {
    Serial.println("Please upgrade the firmware");
  }
```

```
  // by default the local IP address will be 192.168.4.1
  // you can override it with the following:
  WiFi.config(IPAddress(192,48,56,2));

  // print the network name (SSID);
  Serial.print("Creating access point named: ");
  Serial.println(ssid);

  // Create open network
  status = WiFi.beginAP(ssid, pass);
  if (status != WL_AP_LISTENING) {
    Serial.println("Creating access point failed");
    // don't continue
    while (true);
  }
  Serial.println("Waiting for connection");
  // wait 10 seconds for connection:
  delay(10000);

  // start the web server on port 80
  server.begin();

  // you're connected now, so print out the status
  printWiFiStatus();

  // Initialize the MPU6050 sensor (accelerometer and gyroscope)
  initializeMPU6050();

  // Enable I2C bypass on MPU6050 to directly access the QMC5883L magnetometer
  mpu.setI2CBypass(true);

  // Initialize the QMC5883L magnetometer sensor
  initializeQMC5883L();

  // Initialize BMP180 barometric sensor
  initializeBMP180();
}

void loop() {

  // compare the previous status to the current status
  if (status != WiFi.status()) {
    // it has changed update the variable
    status = WiFi.status();
```

```
  if (status == WL_AP_CONNECTED) {
    // a device has connected to the AP
    Serial.println("Device connected to AP");
  } else {
    // a device has disconnected from the AP, and we are back in listening mode
    Serial.println("Device disconnected from AP");
  }
}

WiFiClient client = server.available();   // listen for incoming clients
WiFiClient outgoing;

if (client) {                             // if you get a client,
  doc.clear();
  Serial.println("new client");           // print a message out the serial port
  String currentLine = "";                // make a String to hold incoming data from the client
  while (client.connected()) {            // loop while the client's connected
    delayMicroseconds(10);                // This is required for the Arduino Nano RP2040 Connect -
otherwise it will loop so fast that SPI will never be served.
    if (client.available()) {             // if there's bytes to read from the client,

      // read GET request for LED input values
      String req = client.readStringUntil('\r');
      Serial.println(req);
      while (client.available()) client.read(); // Flush

      // Default LED values
      int redVal = 0;
      bool blueVal = 0;

      // Parse red and blue values from the GET URL
      int blueIndex = req.indexOf("blue=");
      if (blueIndex != -1) {
        blueVal = req.substring(blueIndex + 5).startsWith("1");
      }

      int redIndex = req.indexOf("red=");
      if (redIndex != -1) {
        redVal = req.substring(redIndex + 4).toInt();
      }

      // Apply LED states
      if (blueVal == 1) {
        digitalWrite(blue, HIGH);
      } else {
        digitalWrite(blue, LOW);
```

```
      }

      analogWrite(red, redVal);

      // Prepare and send sensor data
      JsonDocument doc;
      gatherSensorData(doc);
      doc["red"] = redVal;
      doc["blue"] = blueVal;

      client.println("HTTP/1.1 200 OK");
      client.println("Content-Type: application/json");
      client.println();
      serializeJson(doc, client);
      break;
    }
   }

   client.stop();
   Serial.println("client disconnected");
  }

  // Send POST every 10 seconds
  if (millis() - lastPost >= postInterval) {
   lastPost = millis();
   sendDataToNodeRED();
  }
}

// gather the data from each of the sensors and store it in the json doc
void gatherSensorData(JsonDocument& doc) {
  float duration_us, distance_cm;

  // Ultrasonic
  digitalWrite(TRIG_PIN, HIGH);
  delayMicroseconds(10);
  digitalWrite(TRIG_PIN, LOW);
  duration_us = pulseIn(ECHO_PIN, HIGH);
  distance_cm = 0.017 * duration_us;
  doc["distance"] = distance_cm;

  printMPU6050(doc);
  printQMC5883L(doc);
  printBMP180(doc);
}
```

```
// send the json doc to Node-RED as a POST request
void sendDataToNodeRED() {
  JsonDocument doc;
  gatherSensorData(doc);

  String payload;
  serializeJson(doc, payload);
  Serial.println("Sending POST request");
  Serial.println(payload);

  WiFiClient outgoing;
  if (outgoing.connect(nodeRedIP, nodeRedPort)) {
    outgoing.println("POST /ledcontrol HTTP/1.1");
    outgoing.println("Host: 192.48.56.3");
    outgoing.println("Content-Type: application/json");
    outgoing.print("Content-Length: ");
    outgoing.println(payload.length());
    outgoing.println();
    outgoing.println(payload);

    outgoing.stop();
  } else {
    Serial.println("Failed to connect");
  }
}

// initialize and calibrate compass
void initializeQMC5883L() {

  compass.init();

  // calibrations from compass_calibration.ino
  compass.setCalibrationOffsets(-419.00, 1789.00, 4861.00);
  compass.setCalibrationScales(1.18, 1.73, 0.64);

}

// measure and add azimuth and direction values to json doc
void printQMC5883L(JsonDocument& doc) {

  int x, y, z, a;
  char myArray[3];

  compass.read();

  a = compass.getAzimuth();
```

```
  compass.getDirection(myArray, a);

  doc["azimuth"] = a;

  doc["direction"] = myArray;
}

void initializeMPU6050() {
  // Check if the MPU6050 sensor is detected
  if (!mpu.begin()) {
    Serial.println("Failed to find MPU6050 chip");
    while (1)
      ;  // Halt if sensor not found
  }

  // set accelerometer range to +-8G
  mpu.setAccelerometerRange(MPU6050_RANGE_8_G);

  // set gyro range to +- 500 deg/s
  mpu.setGyroRange(MPU6050_RANGE_500_DEG);

  // set filter bandwidth to 21 Hz
  mpu.setFilterBandwidth(MPU6050_BAND_21_HZ);

  delay(100);
}

// measure and add acceleration and rotation values to json doc
void printMPU6050(JsonDocument& doc) {

  sensors_event_t a, g, temp;
  mpu.getEvent(&a, &g, &temp);

  doc["accelerationX"] = a.acceleration.x;
  doc["accelerationY"] = a.acceleration.y;
  doc["accelerationZ"] = a.acceleration.z;

  doc["rotationX"] = g.gyro.x;
  doc["rotationY"] = g.gyro.y;
  doc["rotationZ"] = g.gyro.z;
}

void initializeBMP180() {
  // Start BMP180 initialization
  if (!bmp.begin()) {
```

```
    Serial.println("Could not find a valid BMP180 sensor, check wiring!");
    while (1)
      ;  // Halt if sensor not found
  }
}

// measure and add temp, pressure, altitude, and sea level pressure values to json doc
void printBMP180(JsonDocument& doc) {

  doc["temperature"] = bmp.readTemperature();

  doc["pressure"] = bmp.readPressure();

  // Calculate altitude assuming 'standard' barometric
  // pressure of 1013.25 millibar = 101325 Pascal
  doc["altitude"] = bmp.readAltitude();

  doc["sealevelpressure"] = bmp.readSealevelPressure();
}

void printWiFiStatus() {
  // print the SSID of the network you're attached to:
  Serial.print("SSID: ");
  Serial.println(WiFi.SSID());

  // print your WiFi shield's IP address:
  IPAddress ip = WiFi.localIP();
  Serial.print("IP Address: ");
  Serial.println(ip);
}
```

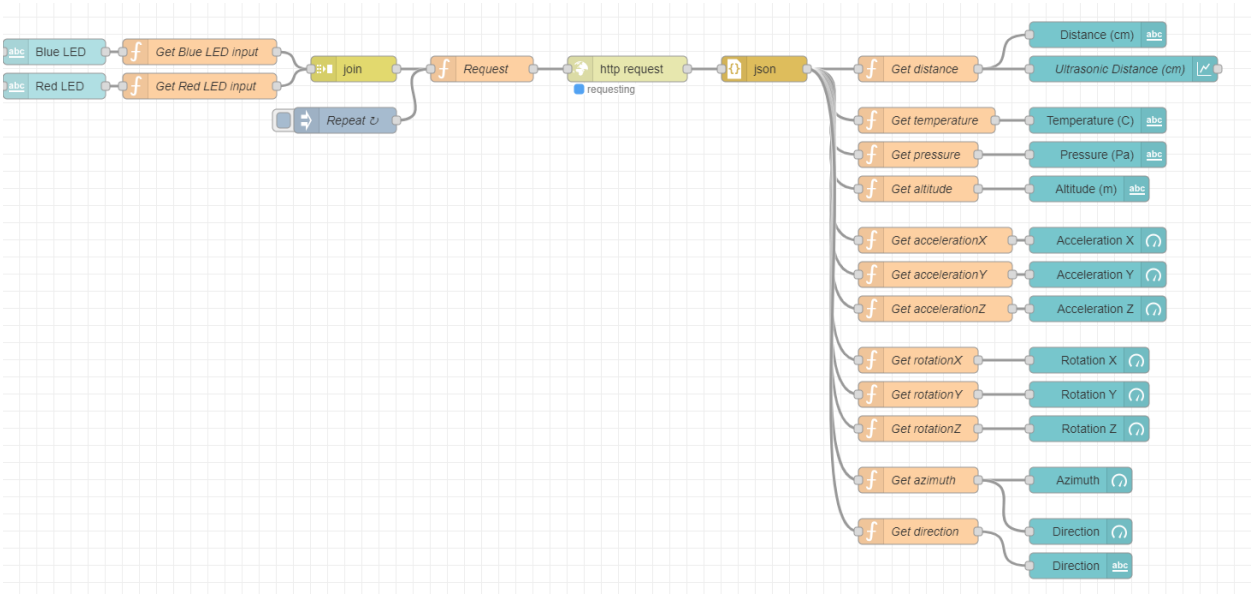
Python code:

```
import requests

def get_response_info(url):
    response = requests.get(url)
    apparent_encoding = response.apparent_encoding
    encoding = response.apparent_encoding
    content = response.content
    headers = response.headers
    status_code = response.status_code
    URL = response.url
    data = response.json() if response.headers.get('Content-Type') == 'application/json' else None
    print(f"URL: {URL}")
    print(f"Apparent Encoding: {apparent_encoding}")
```

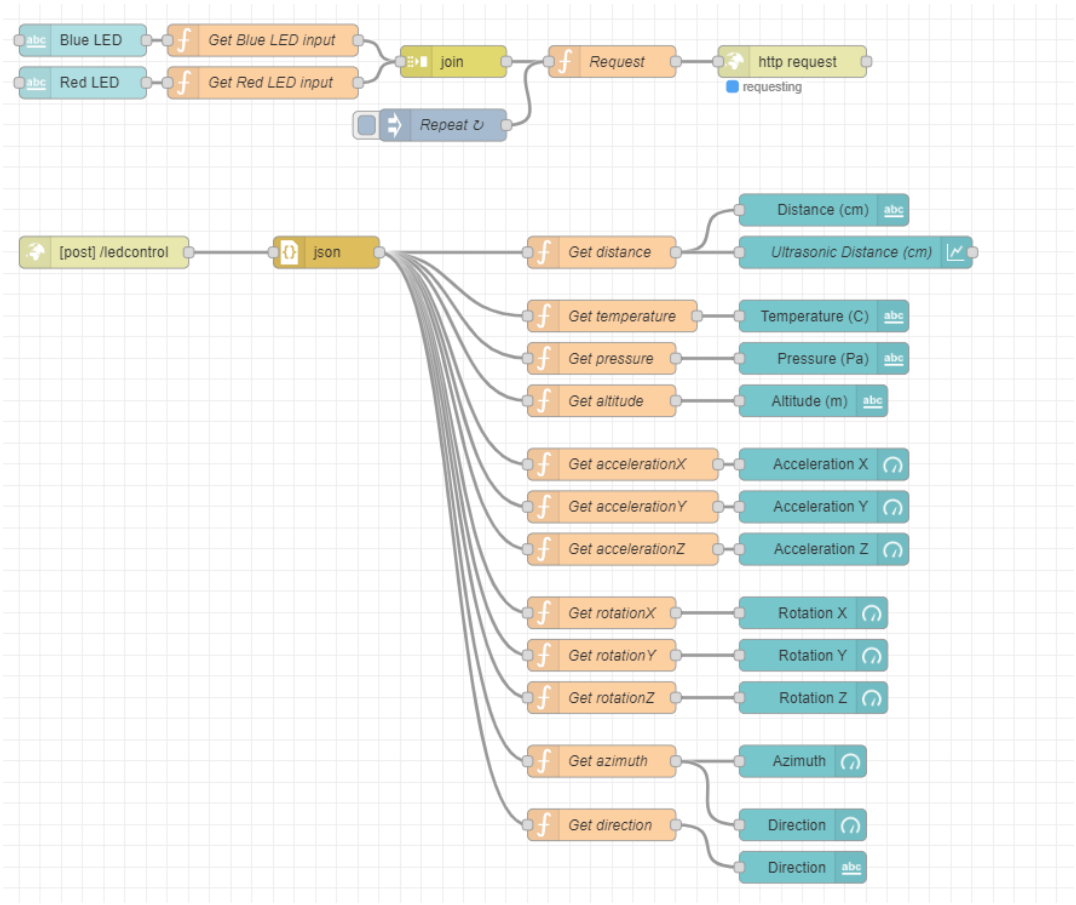
```
    print(f"Encoding: {encoding}")
    print(f"Content: {content}")
    print(f"Headers: {headers}")
    print(f"Status Code: {status_code}")
    print(f"Data: {data}")

blue = input('Enter the blue value (0-1): ')
red = input('Enter the red value (0-255): ')

url = f'http://192.48.56.2/blue={blue}&red={red}'

a = input(f'Press ENTER to Send Request to {url}')
get_response_info(url)
# The code above is a Python script that sends an HTTP GET request to a specified URL to affect
LED bahavior and prints various
# information about the response.
```

Node-Red flows:

(Part A)

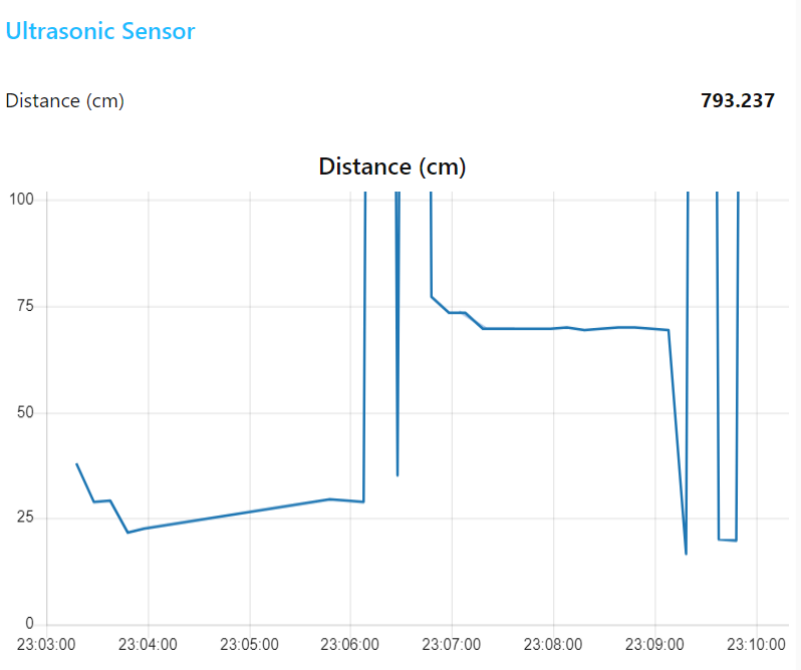


(Part B)



Node-Red GUIs:

(Part A)

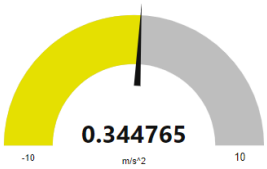


Temperature, Pressure, and Altitude Table

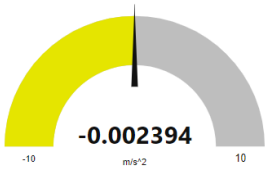
Temperature (C)	**27.6**	Pressure (Pa)	**101258**	Altitude (m)	**5.746357**

Acceleration Gauge

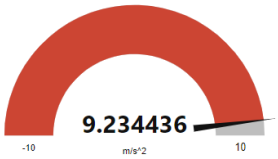
Acceleration X



Acceleration Y

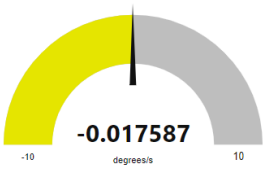


Acceleration Z

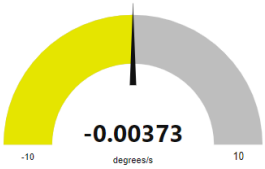


Rotation Gauge

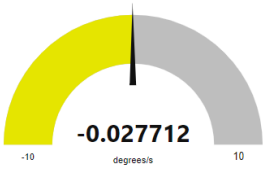
Rotation X



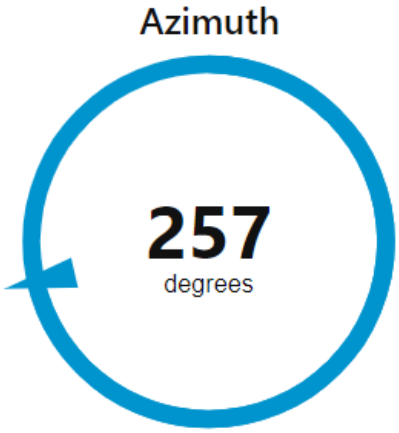
Rotation Y

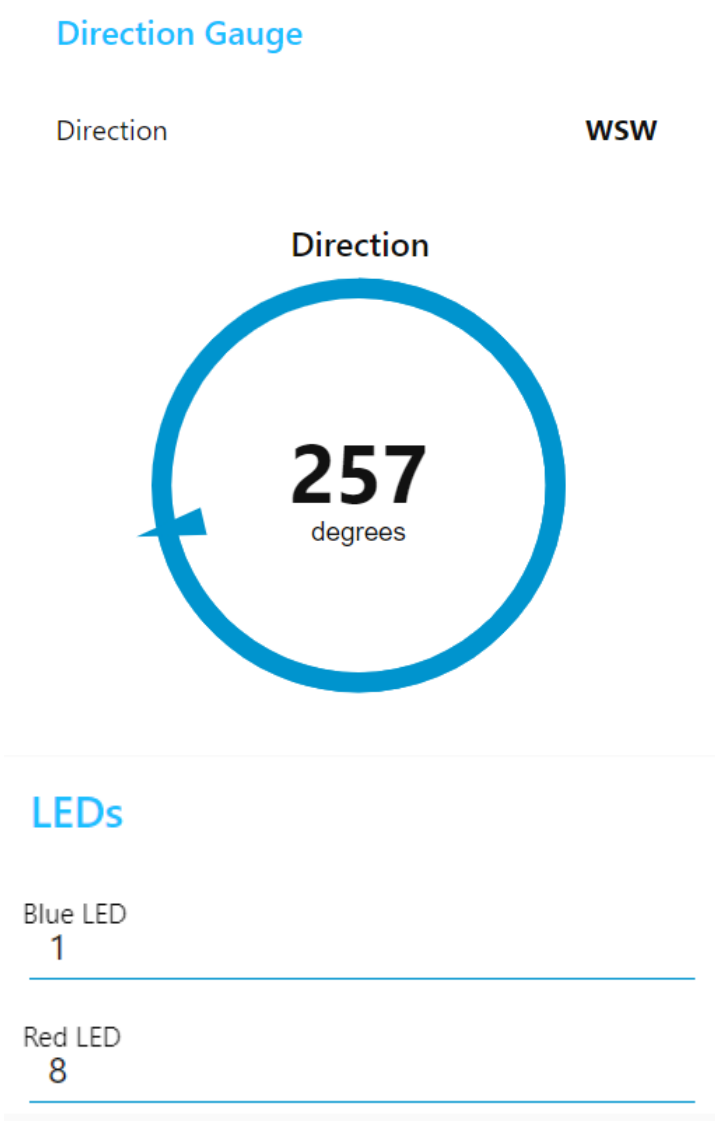


Rotation Z



Azimuth Gauge

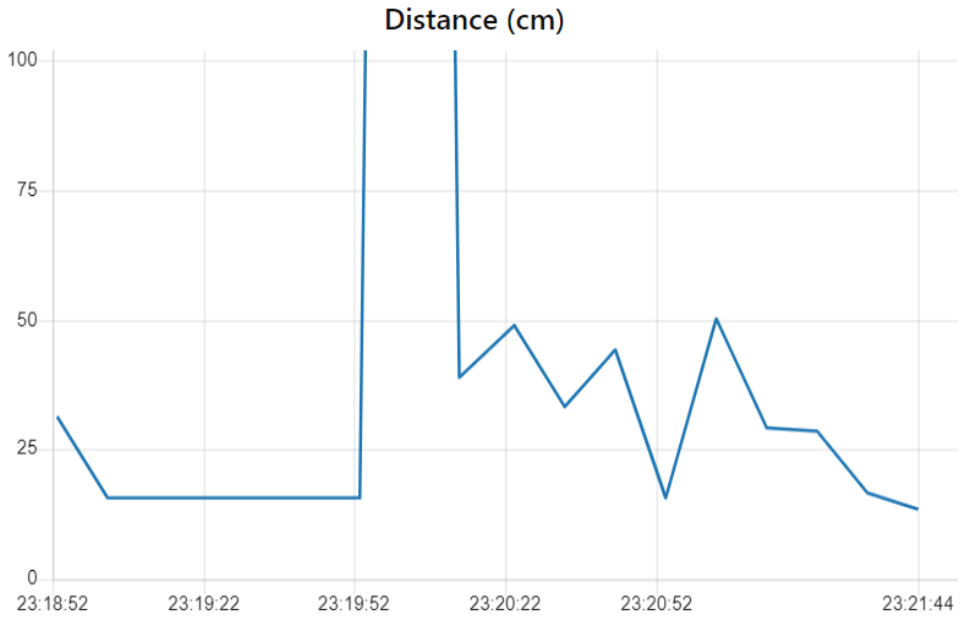



Direction Gauge
Direction
WSW
Direction
257
degrees
LEDs
Blue LED
1
Red LED
8

(Part B)

Ultrasonic Sensor

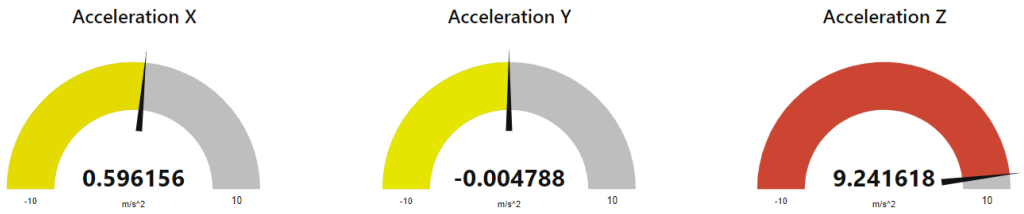
Distance (cm) **13.43**



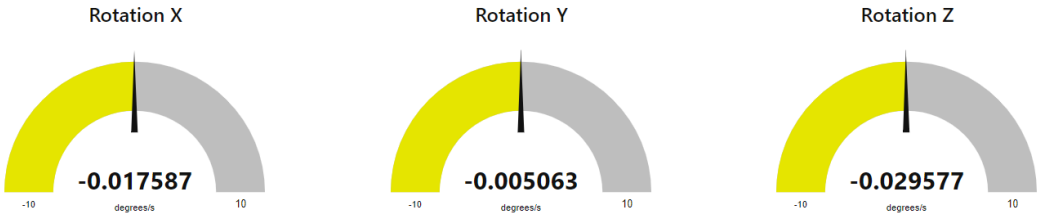
Temperature, Pressure, and Altitude Table

Temperature (C) **27.5** Pressure (Pa) **101257** Altitude (m) **5.829369**

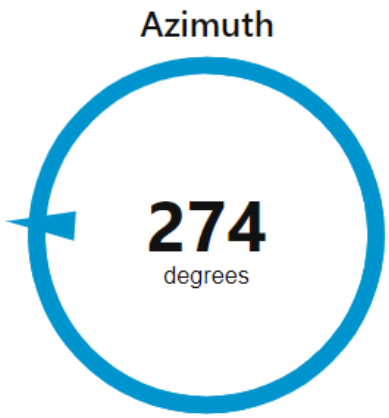
Acceleration Gauge



Rotation Gauge



Azimuth Gauge



Direction Gauge

Direction W

LEDs

Blue LED
1

Red LED
67

Subject: Assignment 6 – Station Mode

To: Christopher Peters, PhD.
Teaching Professor, Electrical and Computer Engineering Department
Drexel University

From: Sofia D'Alessandro
Undergraduate, College of Engineering
Drexel University

Purpose

The purpose of this memo is to demonstrate implementing GET and POST requests using wireless communication to affect the behavior of LEDs after placing the Arduino in Station Mode. This project is almost exactly the same as Assignment 5 except the Arduino was placed in AP Mode then. This assignment required the Arduino to be placed in Station Mode so it could connect to an existing WiFi network and then be controlled by a cell phone that was also connected to that same network. The project was divided into two parts. The first part required the Arduino to act as a wireless server and receive a GET request. The second part required the Arduino to act as a wireless client and send a POST request.

Methodology/Approach

In conducting the lab, the first step was to physically build the circuit that would be used in all three parts. It was the same circuit used in Assignment 5. The circuit was built using a solderless breadboard, the Arduino Uno R4 WiFi, the HC-SR04 ultrasonic sensor, the GY-87 sensor, one red LED, one blue LED, two 1000-ohm resistors, and jumper wires. The ultrasonic sensor was placed on the breadboard. The power rail of the breadboard was connected to the 5V pin on the Arduino and then the ultrasonic sensor's Vcc pin and the GY-87 sensor's Vcc_in pin were connected to the power rail. The ground rail was connected to a ground pin on the Arduino and then the ultrasonic sensor's ground pin was connected to the ground rail. The ultrasonic sensor's trig pin was connected to pin 12 on the Arduino and the ultrasonic sensor's echo pin was connected to pin 13 on the Arduino. The GY-87 sensor's ground pin was connected to a ground pin on the Arduino and the GY-87 sensor's 3.3V pin was connected to the 3.3V pin on the Arduino. The GY-87 sensor's SCL pin was connected to the SCL pin on the Arduino and the GY-87 sensor's SDA pin was connected to the SDA pin on the Arduino. The red and blue LEDS were then placed on the breadboard. Their cathodes were connected to the other ground rail while their anodes were connected to the resistors, which were then connected to pin 6 and pin 5, respectively. Finally, both ground rails were connected.

After the circuit was built, work in the IDE and Node-Red could begin. Two different programs and flows were written and created: one for when the Arduino acted as the server receiving the GET request from Node-RED and the other for when the Arduino acted as the client sending the POST request to Node-RED. The program for Part A was written first. The program was almost exactly the same as its counterpart from Assignment 5, except the Arduino now had to connect to an existing network instead of creating its own access point. The Arduino was also configured

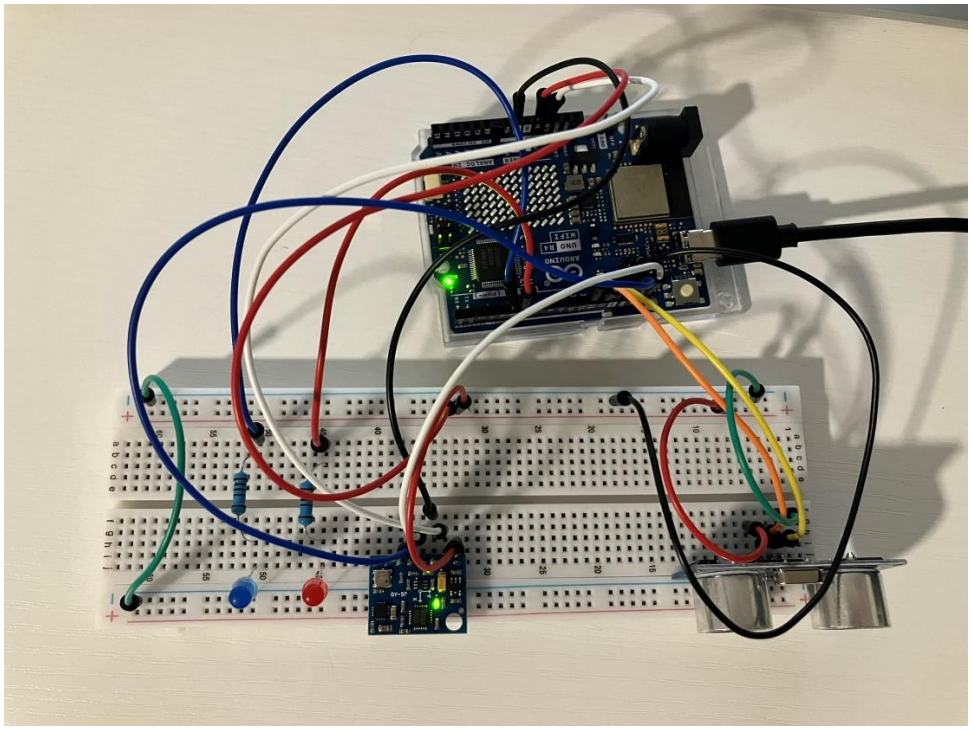
with a static IP address that was not being used by the network. In Node-RED, a flow needed to be created to send the GET request to the Arduino and also to display the results from the response from the Arduino. The exact same flow that was used for Part A in Assignment 5 was used. The only change that had to be made was updating the URL in the Request function node before the http request node so that it had the same IP address that was assigned to the Arduino in the IDE code.

The program for Part B was written next. The program was almost exactly the same as its counterpart from Assignment 5, except the Arduino now had to connect to a network instead of creating its own access point. The Arduino was also configured with a static IP address that was not being used by the network. The Node-RED flow was then created. The exact same flow that was used for Part B in Assignment 5 was used. The only change that had to be made was updating the URL in the Request function node before the http request node so that it had the same IP address that was assigned to the Arduino in the IDE code.

Results

The project was successful. The Arduino was able to connect to an existing WiFi network and was able to be controlled by a cell phone for both parts of the assignment. In Part A, the Arduino was able to receive a GET request from Node-RED with values for the blue and red LEDs which were inputted from the Node-RED UI from a cell phone. In Part B, the Arduino was able to send a POST request to Node-RED with values for the blue and red LEDs which were inputted from the Node-RED UI from a cell phone. In both parts of the assignment, the Arduino was able to turn the blue LED on and off and change the intensity of the red LED based on the parameters given in the requests. The data from the sensors was also able to be sent from the Arduino to Node-RED and was displayed properly in each respective GUI.

Picture of hardware:



Appendix

Arduino code:

(Part A)

```
// Include required libraries
#include <Adafruit_MPU6050.h>
#include <Adafruit_Sensor.h>
#include <Wire.h>
#include <Adafruit_BMP085.h>
#include <QMC5883LCompass.h>
#include <ArduinoJson.h> // Import ArduinoJson library
#include "WiFiS3.h"

char ssid[] = "network1";        // your network SSID (name)
char pass[] = "password123"; // your network password (use for WPA, or use as key for WEP)
int keyIndex = 0;                // your network key index number (needed only for WEP)

int status = WL_IDLE_STATUS;  // Initial Status of WiFi connection

WiFiServer server(80); // Create the web server on port 80

#define TRIG_PIN  12 // Arduino pin connected to sensor TRIGGER pin
#define ECHO_PIN  13 // Arduino pin connected to sensor ECHO pin

float duration_us, distance_cm;
JsonDocument doc;

// Initialize sensor objects
Adafruit_MPU6050 mpu;
Adafruit_BMP085 bmp;
QMC5883LCompass compass;

// initialize pins for each LED
int red = 6;
int blue = 5;

void setup() {
  // Initialize the serial communication with a baud rate of 9600
  Serial.begin(9600);

  while (!Serial) {
    ; // wait for serial port to connect. Needed for native USB port only
  }
  Serial.println("Station Mode Web Server");

  pinMode(TRIG_PIN, OUTPUT); // configure the trigger pin to output mode
```

```
pinMode(ECHO_PIN, INPUT); // configure the echo pin to input mode
pinMode(red, OUTPUT);
pinMode(blue, OUTPUT);

// check for the WiFi module:
if (WiFi.status() == WL_NO_MODULE) {
  Serial.println("Communication with WiFi module failed!");
  // don't continue
  while (true);
}

// check firmware version
String fv = WiFi.firmwareVersion();
if (fv < WIFI_FIRMWARE_LATEST_VERSION) {
  Serial.println("Please upgrade the firmware");
}

// by default the local IP address will be 192.168.4.1
// you can override it with the following:
WiFi.config(IPAddress(192,168,1,179));

// Connect to network
status = WiFi.begin(ssid, pass);
if (status != WL_CONNECTED) {
  Serial.println("Connection to network failed");
  // don't continue
  while (true);
}
Serial.println("Waiting for connection");
// wait 10 seconds for connection:
delay(10000);

Serial.println("Connected to network");
delay(5000);

// start the web server on port 80
server.begin();

// you're connected now, so print out the status
printWiFiStatus();

// Initialize the MPU6050 sensor (accelerometer and gyroscope)
initializeMPU6050();

// Enable I2C bypass on MPU6050 to directly access the QMC5883L magnetometer
mpu.setI2CBypass(true);
```

```
  // Initialize the QMC5883L magnetometer sensor
  initializeQMC5883L();

  // Initialize BMP180 barometric sensor
  initializeBMP180();
}

void loop() {

  // compare the previous status to the current status
  if (status != WiFi.status()) {
    // it has changed update the variable
    status = WiFi.status();

    if (status == WL_AP_CONNECTED) {
      // a device has connected to the AP
      Serial.println("Device connected to AP");
    } else {
      // a device has disconnected from the AP, and we are back in listening mode
      Serial.println("Device disconnected from AP");
    }
  }

  WiFiClient client = server.available();   // listen for incoming clients

  if (client) {                             // if you get a client,
    doc.clear();
    Serial.println("new client");           // print a message out the serial port
    String currentLine = "";                // make a String to hold incoming data from the client
    while (client.connected()) {            // loop while the client's connected
      delayMicroseconds(10);                // This is required for the Arduino Nano RP2040 Connect -
otherwise it will loop so fast that SPI will never be served.
      if (client.available()) {             // if there's bytes to read from the client,
        char c = client.read();             // read a byte, then
        Serial.write(c);                    // print it out to the serial monitor
        if (c == '\n') {                    // if the byte is a newline character

          // if the current line is blank, you got two newline characters in a row.
          // that's the end of the client HTTP request, so send a response:
          if (currentLine.length() == 0) {
            // HTTP headers always start with a response code (e.g. HTTP/1.1 200 OK)
            // and a content-type so the client knows what's coming, then a blank line:
            client.println("HTTP/1.1 200 OK");
            client.println("Content-type: application/json");
            client.println();
```

```
          // The HTTP response ends with another blank line:
          client.println();

          delay (1000);
          // generate 10-microsecond pulse to TRIG pin
          digitalWrite(TRIG_PIN, HIGH);
          delayMicroseconds(10);
          digitalWrite(TRIG_PIN, LOW);

          duration_us = pulseIn(ECHO_PIN, HIGH); // Duration of pulse from ECHO pin
          distance_cm = 0.017 * duration_us; // calculate the distance

          doc["distance"] = distance_cm;

          // Print MPU6050 data
          printMPU6050();

          // Print QMC5883L data
          printQMC5883L();

          // Print BMP180 data
          printBMP180();

          serializeJson(doc, client);
          Serial.println();

          delay(500);

          // break out of the while loop:
          break;
        }
        else {

          if (currentLine.startsWith("GET /blue=1")) {
            digitalWrite(blue, HIGH);  // GET /blue=1 turns the blue LED on
            int index = currentLine.indexOf("red="); // find index of red=
            int intensity = currentLine.substring(index+4).toInt(); // find value of intensity for red
LED given in request
            analogWrite(red, intensity); // change the intensity of the red LED
          }

          if (currentLine.startsWith("GET /blue=0")) {
            digitalWrite(blue, LOW);  // GET /blue=0 turns the blue LED off
            int index = currentLine.indexOf("red="); // find index of red=
```

```
            int intensity = currentLine.substring(index+4).toInt(); // find value of intensity for red
LED given in request
            analogWrite(red, intensity); // change the intensity of the red LED
           }

           // if you got a newline, then clear currentLine:
           currentLine = "";
         }
       } else if (c != '\r') {    // if you got anything else but a carriage return character,
         currentLine += c;      // add it to the end of the currentLine
       }
     }
    }

    client.stop();
    Serial.println("client disconnected");
  }
}

// initialize and calibrate compass
void initializeQMC5883L() {

  compass.init();

  // calibrations from compass_calibration.ino
  compass.setCalibrationOffsets(-419.00, 1789.00, 4861.00);
  compass.setCalibrationScales(1.18, 1.73, 0.64);

}

// measure and add azimuth and direction values to json doc
void printQMC5883L() {

  int x, y, z, a;
  char myArray[3];

  compass.read();

  a = compass.getAzimuth();

  compass.getDirection(myArray, a);

  doc["azimuth"] = a;

  doc["direction"] = myArray;
}
```

```
void initializeMPU6050() {
  // Check if the MPU6050 sensor is detected
  if (!mpu.begin()) {
    Serial.println("Failed to find MPU6050 chip");
    while (1)
      ;  // Halt if sensor not found
  }

  // set accelerometer range to +-8G
  mpu.setAccelerometerRange(MPU6050_RANGE_8_G);

  // set gyro range to +- 500 deg/s
  mpu.setGyroRange(MPU6050_RANGE_500_DEG);

  // set filter bandwidth to 21 Hz
  mpu.setFilterBandwidth(MPU6050_BAND_21_HZ);

  delay(100);
}

// measure and add acceleration and rotation values to json doc
void printMPU6050() {

  sensors_event_t a, g, temp;
  mpu.getEvent(&a, &g, &temp);

  doc["accelerationX"] = a.acceleration.x;
  doc["accelerationY"] = a.acceleration.y;
  doc["accelerationZ"] = a.acceleration.z;

  doc["rotationX"] = g.gyro.x;
  doc["rotationY"] = g.gyro.y;
  doc["rotationZ"] = g.gyro.z;
}

void initializeBMP180() {
  // Start BMP180 initialization
  if (!bmp.begin()) {
    Serial.println("Could not find a valid BMP180 sensor, check wiring!");
    while (1)
      ;  // Halt if sensor not found
  }
}

// measure and add temp, pressure, altitude, and sea level pressure values to json doc
```

```
void printBMP180() {

  doc["temperature"] = bmp.readTemperature();

  doc["pressure"] = bmp.readPressure();

  // Calculate altitude assuming 'standard' barometric
  // pressure of 1013.25 millibar = 101325 Pascal
  doc["altitude"] = bmp.readAltitude();

  doc["sealevelpressure"] = bmp.readSealevelPressure();
}

void printWiFiStatus() {
  // print the SSID of the network you're attached to:
  Serial.print("SSID: ");
  Serial.println(WiFi.SSID());

  // print your WiFi shield's IP address:
  IPAddress ip = WiFi.localIP();
  Serial.print("IP Address: ");
  Serial.println(ip);

  // print your router's IP address
  Serial.print("Gateway IP: ");
  Serial.println(WiFi.gatewayIP());
}
```

(Part B)

```
// Include required libraries
#include <Adafruit_MPU6050.h>
#include <Adafruit_Sensor.h>
#include <Wire.h>
#include <Adafruit_BMP085.h>
#include <QMC5883LCompass.h>
#include <ArduinoJson.h> // Import ArduinoJson library
#include "WiFiS3.h"

char ssid[] = "network1";        // your network SSID (name)
char pass[] = "password123"; // your network password (use for WPA, or use as key for WEP)
int keyIndex = 0;                // your network key index number (needed only for WEP)

int status = WL_IDLE_STATUS;  // Initial Status of WiFi connection

WiFiServer server(80); // Create the web server on port 80
```

```
#define TRIG_PIN  12 // Arduino pin connected to sensor TRIGGER pin
#define ECHO_PIN  13 // Arduino pin connected to sensor ECHO pin

float duration_us, distance_cm;
JsonDocument doc;

// Initialize sensor objects
Adafruit_MPU6050 mpu;
Adafruit_BMP085 bmp;
QMC5883LCompass compass;

// initialize pins for each LED
int red = 6;
int blue = 5;

// Node-RED IP address and port number
IPAddress nodeRedIP(192, 168, 1, 156);
const int nodeRedPort = 1880;

// Timer
unsigned long lastPost = 0;
const unsigned long postInterval = 10000;  // every 10 seconds

// Static IP configuration
IPAddress localIP(192, 168, 1, 179);     // Set to a free IP on your network
IPAddress gateway(192, 168, 1, 1);       // Your router's IP address
IPAddress subnet(255, 255, 255, 0);      // Subnet mask

void setup() {
  // Initialize the serial communication with a baud rate of 9600
  Serial.begin(9600);

  while (!Serial) {
    ; // wait for serial port to connect. Needed for native USB port only
  }
  Serial.println("Access Point Web Server");

  pinMode(TRIG_PIN, OUTPUT); // configure the trigger pin to output mode
  pinMode(ECHO_PIN, INPUT); // configure the echo pin to input mode
  pinMode(red, OUTPUT);
  pinMode(blue, OUTPUT);

  // check for the WiFi module:
  if (WiFi.status() == WL_NO_MODULE) {
    Serial.println("Communication with WiFi module failed!");
```

```
  // don't continue
  while (true);
 }

 // check firmware version
 String fv = WiFi.firmwareVersion();
 if (fv < WIFI_FIRMWARE_LATEST_VERSION) {
  Serial.println("Please upgrade the firmware");
 }

 // by default the local IP address will be 192.168.4.1
 // you can override it with the following:
 WiFi.config(localIP, gateway, subnet);

 // Connect to network
 status = WiFi.begin(ssid, pass);
 if (status != WL_CONNECTED) {
  Serial.println("Connection to network failed");
  // don't continue
  while (true);
 }
 Serial.println("Waiting for connection");
 // wait 10 seconds for connection:
 delay(10000);

 Serial.println("Connected to network");
 delay(5000);

 // start the web server on port 80
 server.begin();

 // you're connected now, so print out the status
 printWiFiStatus();

 // Initialize the MPU6050 sensor (accelerometer and gyroscope)
 initializeMPU6050();

 // Enable I2C bypass on MPU6050 to directly access the QMC5883L magnetometer
 mpu.setI2CBypass(true);

 // Initialize the QMC5883L magnetometer sensor
 initializeQMC5883L();

 // Initialize BMP180 barometric sensor
 initializeBMP180();
}
```

```
void loop() {

  WiFiClient client = server.available();   // listen for incoming clients
  WiFiClient outgoing;

  if (client) {                             // if you get a client,
    doc.clear();
    Serial.println("new client");           // print a message out the serial port
    String currentLine = "";                // make a String to hold incoming data from the client
    while (client.connected()) {            // loop while the client's connected
      delayMicroseconds(10);                // This is required for the Arduino Nano RP2040 Connect - otherwise it will loop so fast that SPI will never be served.
      if (client.available()) {             // if there's bytes to read from the client,

        // read GET request for LED input values
        String req = client.readStringUntil('\r');
        Serial.println(req);
        while (client.available()) client.read(); // Flush

        // Default LED values
        int redVal = 0;
        bool blueVal = 0;

        // Parse red and blue values from the GET URL
        int blueIndex = req.indexOf("blue=");
        if (blueIndex != -1) {
          blueVal = req.substring(blueIndex + 5).startsWith("1");
        }

        int redIndex = req.indexOf("red=");
        if (redIndex != -1) {
          redVal = req.substring(redIndex + 4).toInt();
        }

        // Apply LED states
        if (blueVal == 1) {
          digitalWrite(blue, HIGH);
        } else {
          digitalWrite(blue, LOW);
        }

        analogWrite(red, redVal);

        // Prepare and send sensor data
        JsonDocument doc;
```

```
        gatherSensorData(doc);
        doc["red"] = redVal;
        doc["blue"] = blueVal;

        client.println("HTTP/1.1 200 OK");
        client.println("Content-Type: application/json");
        client.println();
        serializeJson(doc, client);
        break;
      }
    }

    client.stop();
    Serial.println("client disconnected");
  }

  // Send POST every 10 seconds
  if (millis() - lastPost >= postInterval) {
    lastPost = millis();
    sendDataToNodeRED();
  }
}

// gather the data from each of the sensors and store it in the json doc
void gatherSensorData(JsonDocument& doc) {
  float duration_us, distance_cm;

  // Ultrasonic
  digitalWrite(TRIG_PIN, HIGH);
  delayMicroseconds(10);
  digitalWrite(TRIG_PIN, LOW);
  duration_us = pulseIn(ECHO_PIN, HIGH);
  distance_cm = 0.017 * duration_us;
  doc["distance"] = distance_cm;

  printMPU6050(doc);
  printQMC5883L(doc);
  printBMP180(doc);
}

// send the json doc to Node-RED as a POST request
void sendDataToNodeRED() {
  JsonDocument doc;
  gatherSensorData(doc);

  String payload;
```

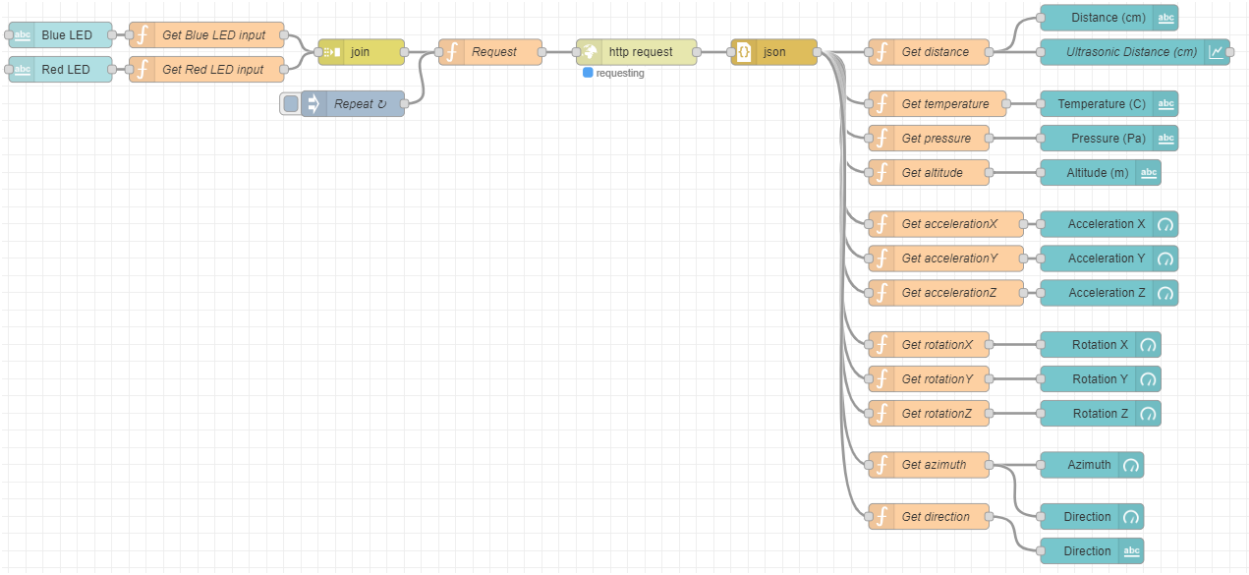
```
  serializeJson(doc, payload);
  Serial.println("Sending POST request");
  Serial.println(payload);

  WiFiClient outgoing;
  if (outgoing.connect(nodeRedIP, nodeRedPort)) {
    outgoing.println("POST /ledcontrol HTTP/1.1");
    outgoing.println("Host: 192.168.1.156");
    outgoing.println("Content-Type: application/json");
    outgoing.print("Content-Length: ");
    outgoing.println(payload.length());
    outgoing.println();
    outgoing.println(payload);

    outgoing.stop();
  } else {
    Serial.println("Failed to connect");
  }
}

// initialize and calibrate compass
void initializeQMC5883L() {

  compass.init();

  // calibrations from compass_calibration.ino
  compass.setCalibrationOffsets(-419.00, 1789.00, 4861.00);
  compass.setCalibrationScales(1.18, 1.73, 0.64);

}

// measure and add azimuth and direction values to json doc
void printQMC5883L(JsonDocument& doc) {

  int x, y, z, a;
  char myArray[3];

  compass.read();

  a = compass.getAzimuth();

  compass.getDirection(myArray, a);

  doc["azimuth"] = a;

  doc["direction"] = myArray;
```

```
}

void initializeMPU6050() {
  // Check if the MPU6050 sensor is detected
  if (!mpu.begin()) {
    Serial.println("Failed to find MPU6050 chip");
    while (1)
      ;  // Halt if sensor not found
  }

  // set accelerometer range to +-8G
  mpu.setAccelerometerRange(MPU6050_RANGE_8_G);

  // set gyro range to +- 500 deg/s
  mpu.setGyroRange(MPU6050_RANGE_500_DEG);

  // set filter bandwidth to 21 Hz
  mpu.setFilterBandwidth(MPU6050_BAND_21_HZ);

  delay(100);
}

// measure and add acceleration and rotation values to json doc
void printMPU6050(JsonDocument& doc) {

  sensors_event_t a, g, temp;
  mpu.getEvent(&a, &g, &temp);

  doc["accelerationX"] = a.acceleration.x;
  doc["accelerationY"] = a.acceleration.y;
  doc["accelerationZ"] = a.acceleration.z;

  doc["rotationX"] = g.gyro.x;
  doc["rotationY"] = g.gyro.y;
  doc["rotationZ"] = g.gyro.z;
}

void initializeBMP180() {
  // Start BMP180 initialization
  if (!bmp.begin()) {
    Serial.println("Could not find a valid BMP180 sensor, check wiring!");
    while (1)
      ;  // Halt if sensor not found
  }
}
```

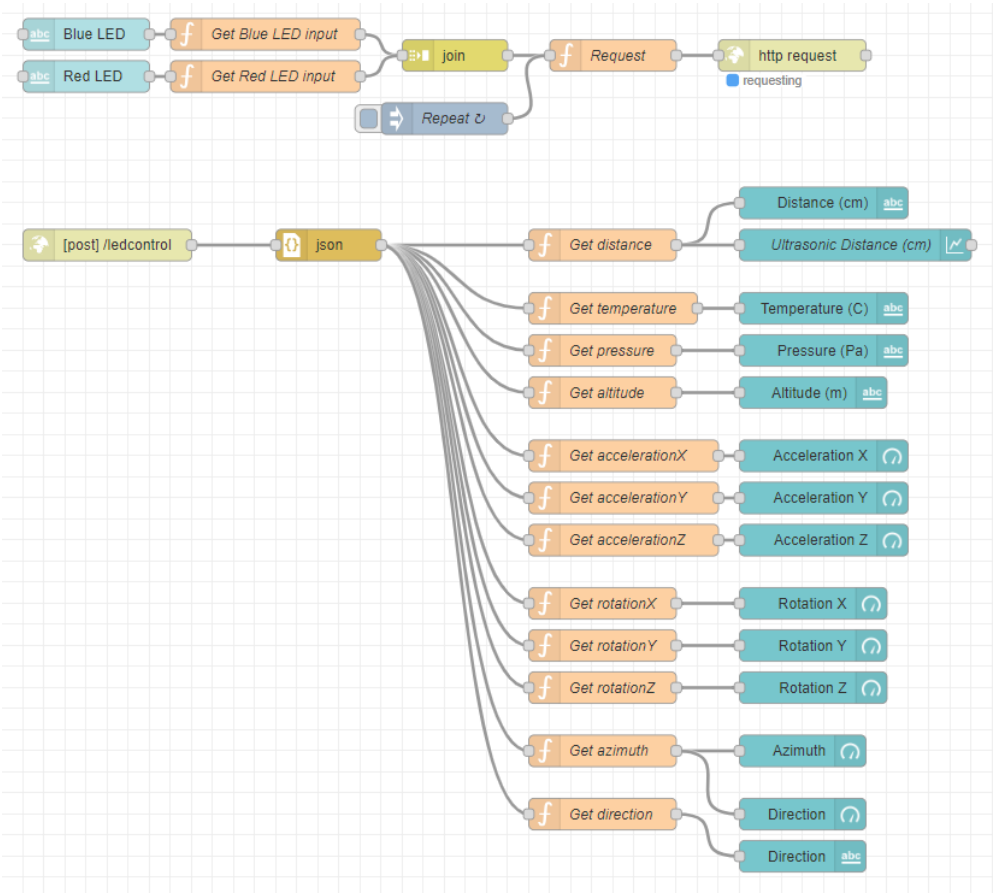
```
// measure and add temp, pressure, altitude, and sea level pressure values to json doc
void printBMP180(JsonDocument& doc) {

  doc["temperature"] = bmp.readTemperature();

  doc["pressure"] = bmp.readPressure();

  // Calculate altitude assuming 'standard' barometric
  // pressure of 1013.25 millibar = 101325 Pascal
  doc["altitude"] = bmp.readAltitude();

  doc["sealevelpressure"] = bmp.readSealevelPressure();
}

void printWiFiStatus() {
  // print the SSID of the network you're attached to:
  Serial.print("SSID: ");
  Serial.println(WiFi.SSID());

  // print your WiFi shield's IP address:
  IPAddress ip = WiFi.localIP();
  Serial.print("IP Address: ");
  Serial.println(ip);

  // print your router's IP address
  Serial.print("Gateway IP: ");
  Serial.println(WiFi.gatewayIP());

  // print your subnet mask
  Serial.print("Subnet Mask: ");
  Serial.println(WiFi.subnetMask());
}
```

Node-Red flows:

(Part A)



(Part B)

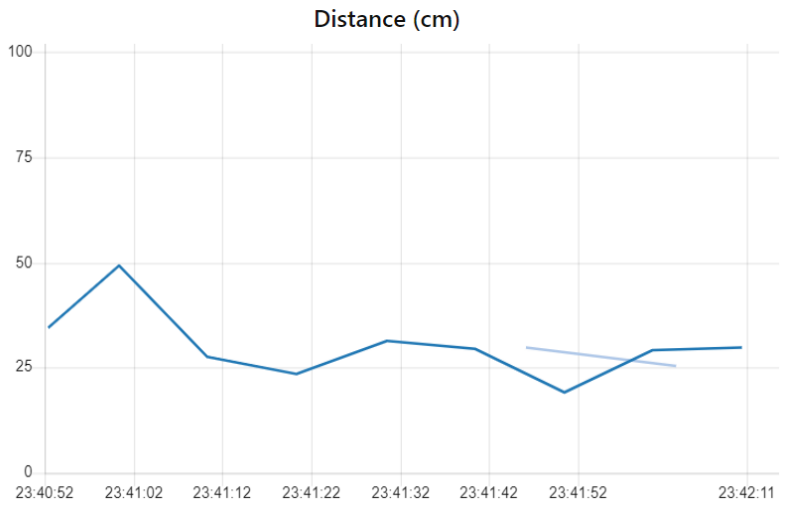


Node-Red GUIs:

(Part A)

Ultrasonic Sensor

Distance (cm) **29.699**

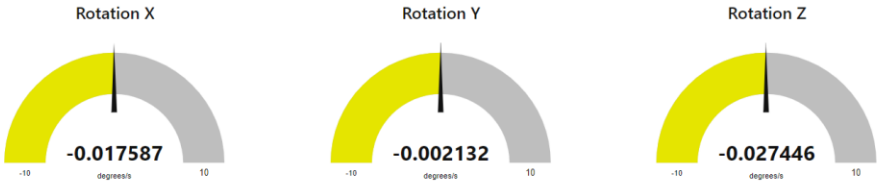


Temperature, Pressure, and Altitude Table

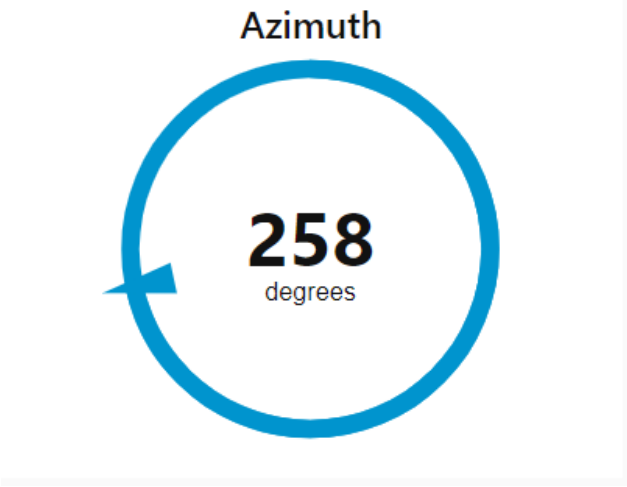
Temperature (C) **28.7** Pressure (Pa) **101540** Altitude (m) **-17.71855**

Acceleration Gauge

Rotation Gauge



Azimuth Gauge



Direction Gauge

Direction **WSW**

LEDs

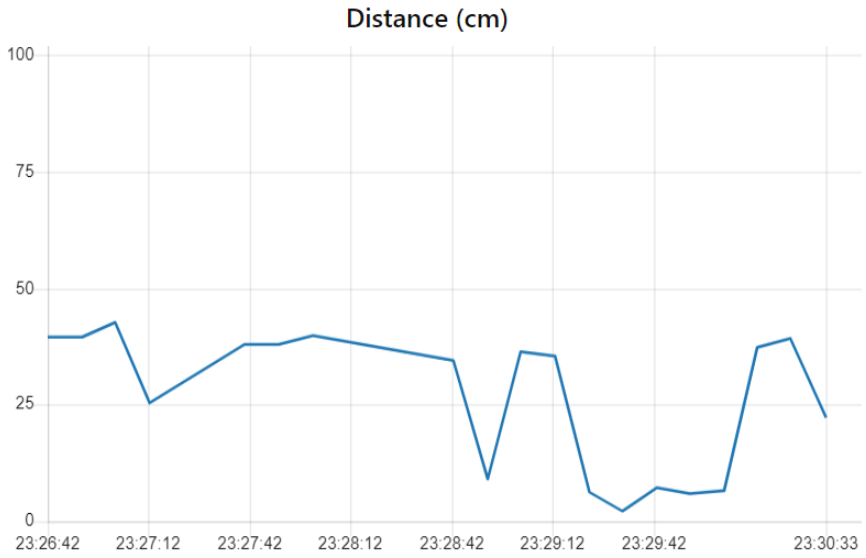
Blue LED
0

Red LED
50

(Part B)

Ultrasonic Sensor

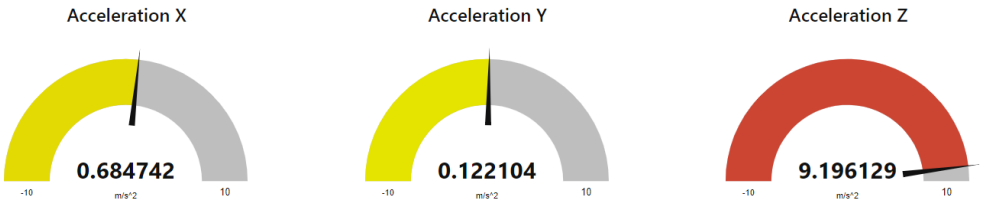
Distance (cm) **22.355**



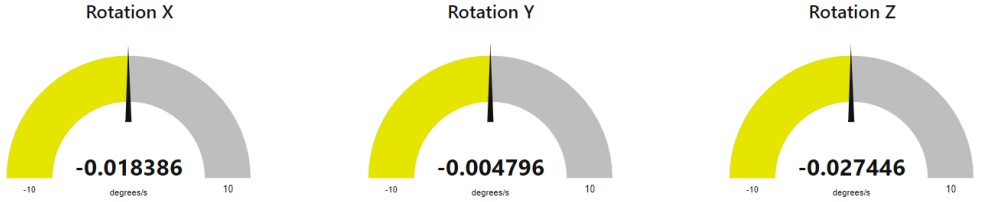
Temperature, Pressure, and Altitude Table

Temperature (C) **29.7** Pressure (Pa) **101557** Altitude (m) **-19.38094**

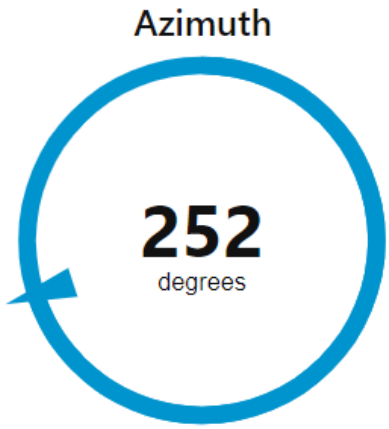
Acceleration Gauge



Rotation Gauge



Azimuth Gauge



Direction Gauge

Direction **WSW**

LEDs

Blue LED
1

Red LED
30